

中国科学院大学
University of Chinese Academy of Sciences

Generative Models & AI4S Applications

Lecture 3 Autoregressive Models

Fall 2026

Liu Yunfan (刘云帆)

liuyunfan@ucas.ac.cn

Announcements

- A make-up lecture will be held on **Sept. 20 (Sun.)** due to the adjusted workday schedule for National Holiday

- **Assignment 1** will be released this weak and due after the National Holiday

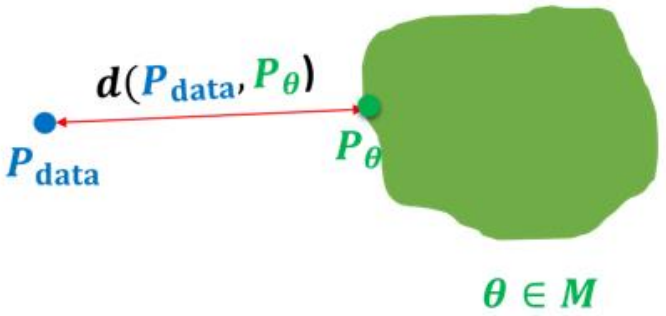
- This slides is subject to future modification, and the updated version will be posted online

Recap: Principles of Statistical Machine Learning

Images of dogs



$x_i \sim P_{\text{data}}$
$i = 1, 2, \ldots, n$



Dog Image Generator

- Three main components of a Statistical ML algorithm:
  - **Model Family:** Define the scope of learnable function (i.e., $P_\theta(x)$)
  - **Learning Objective:** Quantify the target of learning process $\mathcal{L}(x; \theta)$
  - **Optimization:** Algorithm for solving the optimal parameters θ^*
- **Question: How to represent the model as probability $p_\theta(x)$?**

Recap: Modeling of Probability

- **Chain Rule**

$$p(x_1, x_2, x_3, x_4) = p(x_1)\, p(\, x_2 \mid x_1\,)\, p(\, x_3 \mid x_1, x_2\,)\, p(\, x_4 \mid x_1, x_2, x_3\,)$$

  - Fully general, no assumption need (exponential size, no free lunch)

Recap: Modeling of Probability

- **Chain Rule**

$$p(x_1, x_2, x_3, x_4) = p(x_1)\, p(\, x_2 \mid x_1\,)\, p(\, x_3 \mid x_1, x_2\,)\, p(\, x_4 \mid x_1, x_2, x_3\,)$$

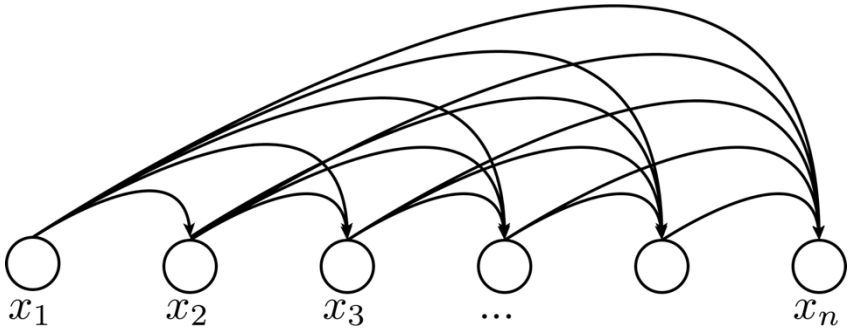
  - Fully general, no assumption need (exponential size, no free lunch)

- **Bayes Networks**

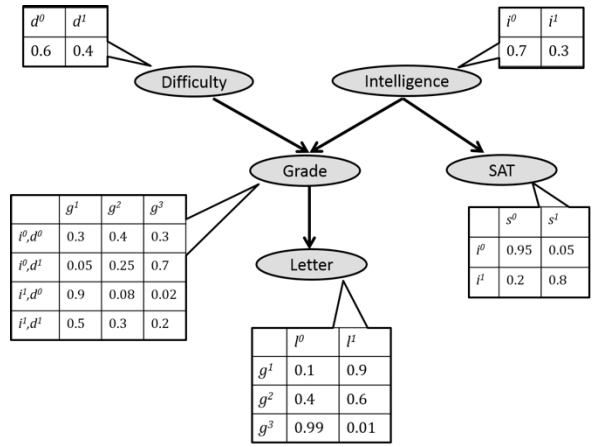
$$p(x_1, x_2, x_3, x_4) \approx p_{CPT}(x_1)\, p_{CPT}(\, x_2 \mid x_1\,)\, p_{CPT}(\, x_3 \mid \cancel{x_1}, x_2\,)\, p_{CPT}(\, x_4 \mid x_1, \cancel{x_2}, \cancel{x_3}\,)$$

  - Assumes conditional independencies ➔ simplify CPT

Complete DAG



CPT: Conditional Probability Table



Recap: Modeling of Probability

- **Chain Rule**

$$p(x_1, x_2, x_3, x_4) = p(x_1)\, p(\, x_2 \mid x_1\,)\, p(\, x_3 \mid x_1, x_2\,)\, p(\, x_4 \mid x_1, x_2, x_3\,)$$

  - Fully general, no assumption need (exponential size, no free lunch)

- **Bayes Networks**

$$p(x_1, x_2, x_3, x_4) \approx p_{CPT}(x_1)\, p_{CPT}(\, x_2 \mid x_1\,)\, p_{CPT}(\, x_3 \mid \cancel{x_1}, x_2\,)\, p_{CPT}(\, x_4 \mid x_1, \cancel{x_2}, \cancel{x_3}\,)$$

  - Assumes conditional independencies ➔ simplify CPT
  - **Limitations:**
    - ① Inter-relationships are **unknown / implicit**
    - ② CPT can be **extremely large** with many variables

Recap: Modeling of Probability

- **Chain Rule**

$$p(x_1, x_2, x_3, x_4) = p(x_1)\, p(\, x_2 \mid x_1\,)\, p(\, x_3 \mid x_1, x_2\,)\, p(\, x_4 \mid x_1, x_2, x_3\,)$$

  - Fully general, no assumption need (exponential size, no free lunch)

- **Bayes Networks**

$$p(x_1, x_2, x_3, x_4) \approx p_{CPT}(x_1)\, p_{CPT}(\, x_2 \mid x_1\,)\, p_{CPT}(\, x_3 \mid \cancel{x_1}, x_2\,)\, p_{CPT}(\, x_4 \mid x_1, \cancel{x_2}, \cancel{x_3}\,)$$

  - Assumes conditional independencies ➔ simplify CPT

- **Neural Networks**

$$p(x_1, x_2, x_3, x_4) \approx p(x_1)\, p(\, x_2 \mid x_1\,)\, p_{Neural}(\, x_3 \mid x_1, x_2\,)\, p_{Neural}(\, x_4 \mid x_1, x_2, x_3\,)$$

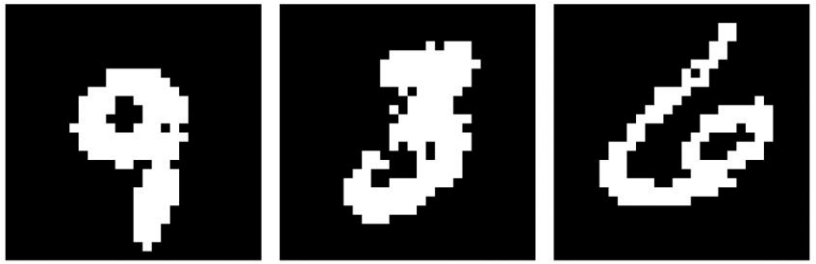
  - **Parametrize** the conditionals, instead of simplify them (**no assumption**)

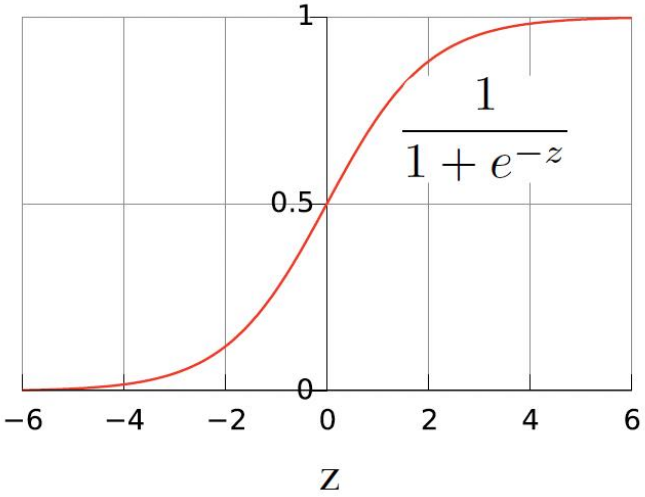
Recap: Modeling of Probability

- **Parameterized Probability Representation**
  - **Linear Dependence:** In logistic regression, we assume

$$p(Y = 1 \mid \boldsymbol{x}; \boldsymbol{\alpha}) = \sigma\big(z(\boldsymbol{x}, \boldsymbol{\alpha})\big)$$

where $z(\boldsymbol{x}, \boldsymbol{\alpha}) = \alpha_0 + \sum \alpha_i x_i$





$\sigma(\cdot)$

- **Neural Networks**

$$p(x_1, x_2, x_3, x_4) \approx p(x_1)\, p(x_2 \mid x_1)\, p_{Neural}(x_3 \mid x_1, x_2)\, p_{Neural}(x_4 \mid x_1, x_2, x_3)$$

  - **Parametrize** the conditionals, instead of simplify them (**no assumption**)

Recap: Modeling of Probability

- **Chain Rule**

$$p(x_1, x_2, x_3, x_4) = p(x_1)\, p(\, x_2 \mid x_1\,)\, p(\, x_3 \mid x_1, x_2\,)\, p(\, x_4 \mid x_1, x_2, x_3\,)$$

  - Fully general, no assumption need (exponential size, no free lunch)

- **Bayes Networks**

$$p(x_1, x_2, x_3, x_4) \approx p_{CPT}(x_1)\, p_{CPT}(\, x_2 \mid x_1\,)\, p_{CPT}(\, x_3 \mid \cancel{x_1}, x_2\,)\, p_{CPT}(\, x_4 \mid x_1, \cancel{x_2}, \cancel{x_3}\,)$$

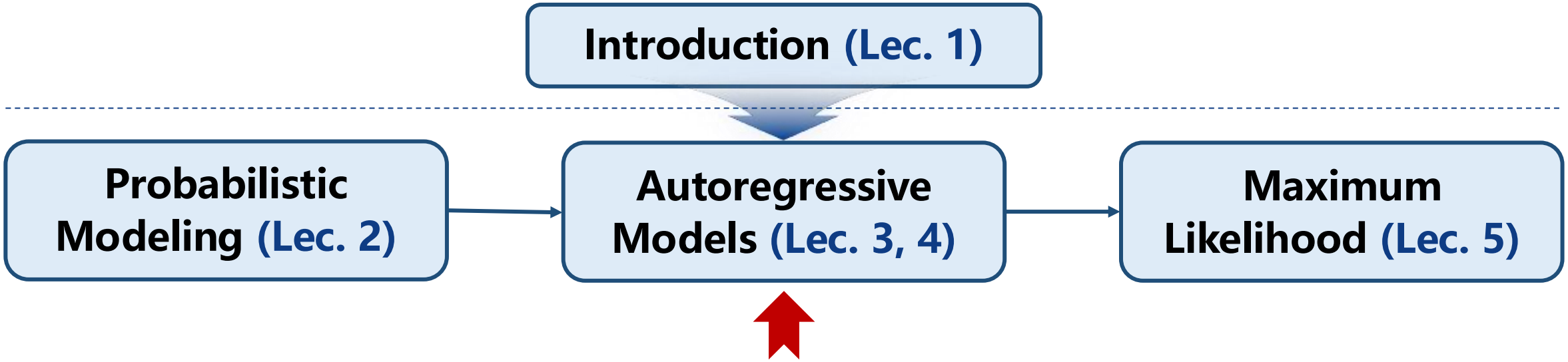
  - Assumes conditional independencies ➔ simplify CPT

- **Neural Networks**

$$p(x_1, x_2, x_3, x_4) \approx p(x_1)\, p(\, x_2 \mid x_1\,)\, p_{Neural}(\, x_3 \mid x_1, x_2\,)\, p_{Neural}(\, x_4 \mid x_1, x_2, x_3\,)$$

  - **Parametrize** the conditionals, instead of simplify them (**no assumption**)

Roadmap



A generative model leveraging the factorization of p_θ



Lecture Outline

- Introduction to Autoregressive Models
- Basic Autoregressive Models
- Network Architectures for Autoregressive Modeling
- Inference and Training Scheme

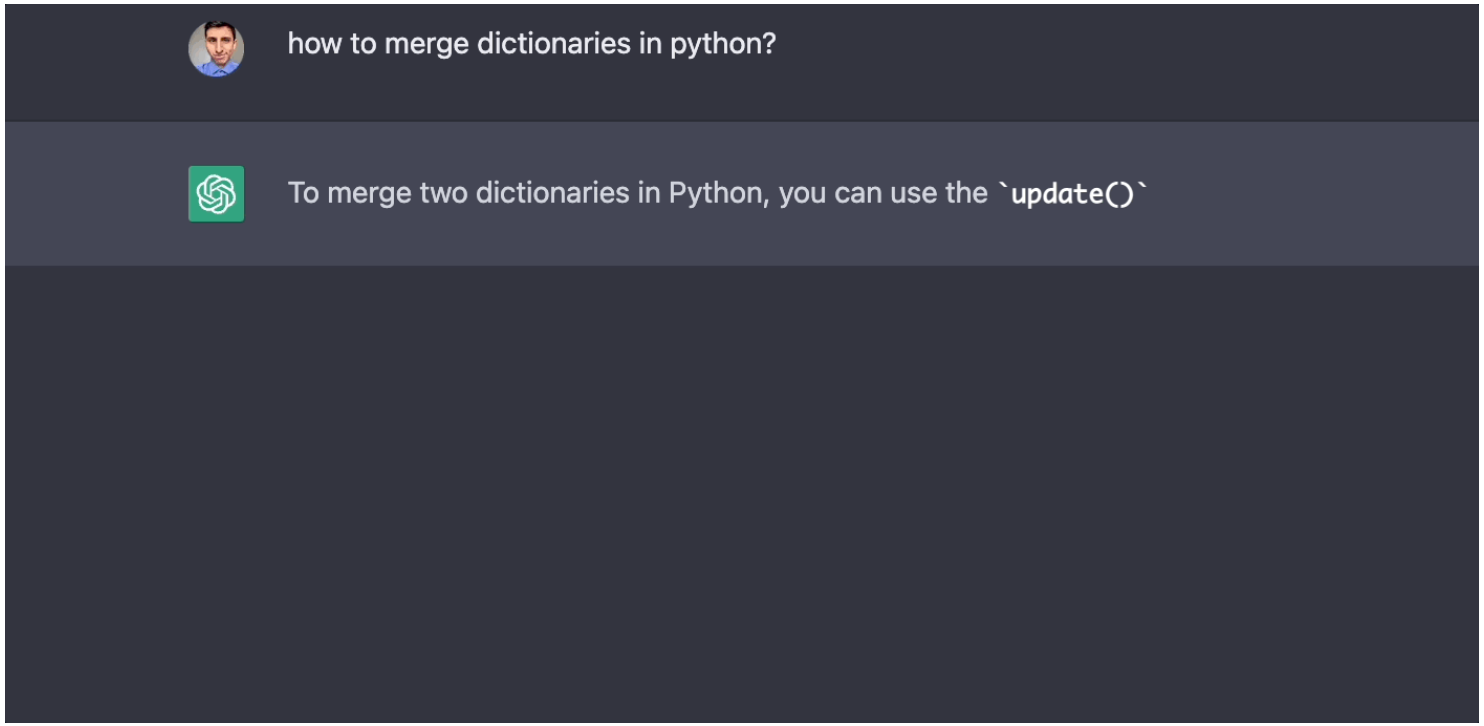


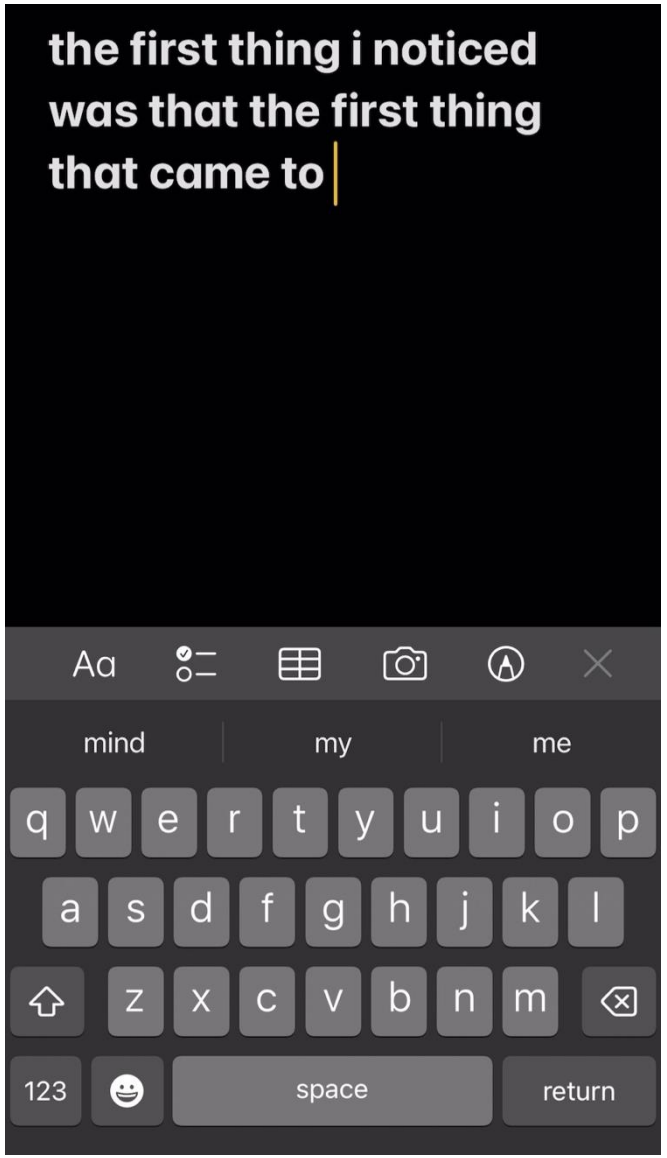
Lecture Outline

- **Introduction to Autoregressive Models**
- **Basic Autoregressive Models**
- **Network Architectures for Autoregressive Modeling**
- **Inference and Training Scheme**

Introduction to Autoregressive Models

- **'Next Token Prediction'**
  - ➢ Existing content can largely determine the next plausible words





Introduction to Autoregressive Models

- **Autoregression (自回归) = Auto + Regression**
  - **Auto: ‘ self ’**
    - ✓ Using its **“own” outputs as inputs** for next predictions
  - **Regression:**
    - ✓ Estimating relationship between variables

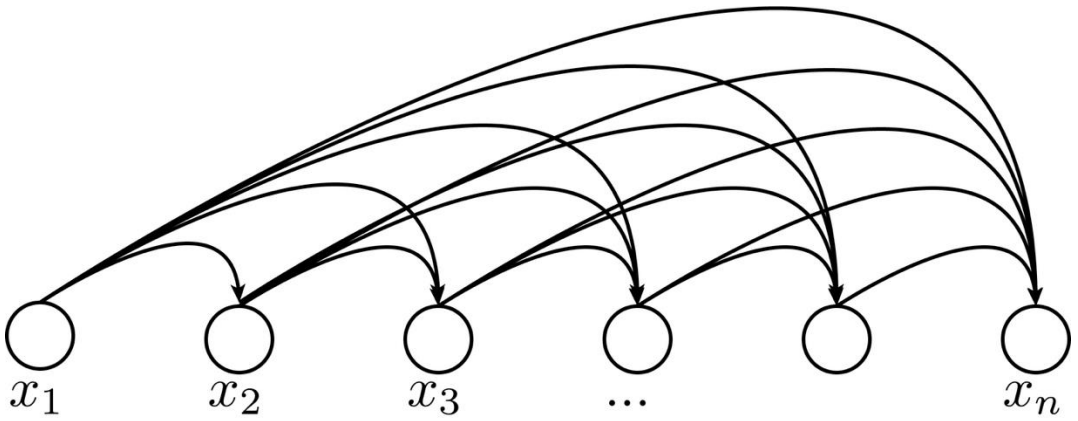
Autoregressive-NESS (AR-NESS)

- **Note:**
  - “Autoregressive” implies an **inference-time (modeling) behavior**
  - Training-time is not necessarily auto-regressive (e.g., teacher-forcing)

Introduction to Autoregressive Models

- **Autoregressive Models**
  - **Autoregression:** A way of modeling joint distribution by **a product of conditional** distributions

$$
\begin{aligned}
p(x_1, x_2, ..., x_n) &= p(x_1)p(x_2 \mid x_1)...p(x_n \mid x_1, x_2, ..., x_{n-1}) \\
&= \prod_{i=1}^{n} p(x_i \mid x_1, x_2, ..., x_{i-1})
\end{aligned}
$$

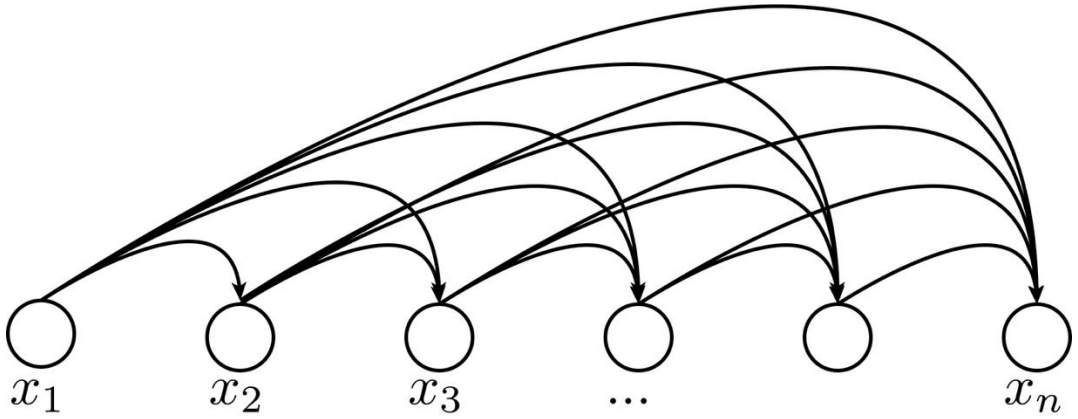


Introduction to Autoregressive Models

- **Autoregressive Models**
  - **Autoregression:** A way of modeling joint distribution by **a product of conditional** distributions

$$
\begin{aligned}
p(x_1, x_2, ..., x_n) &= p(x_1)p(x_2 \mid x_1)...p(x_n \mid x_1, x_2, ..., x_{n-1}) \\
&= \prod_{i=1}^{n} p(x_i \mid x_1, x_2, ..., x_{i-1})
\end{aligned}
$$

  - x can be **any representation**
    - ✓ **Not necessarily sequential**
    - ✓ 2D, 3D, high-dim arrays

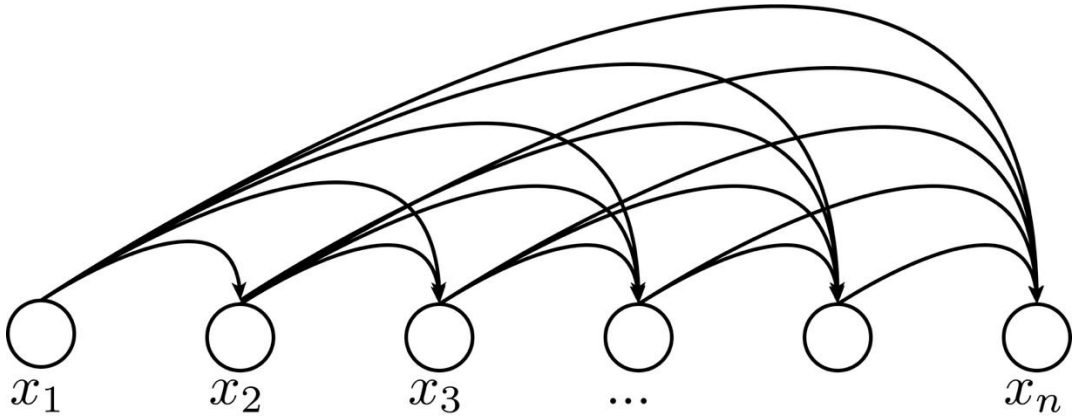


Introduction to Autoregressive Models

- **Autoregressive Models**
  - **Autoregression:** A way of modeling joint distribution by **a product of conditional** distributions

$$
\begin{aligned}
p(x_1, x_2, ..., x_n) &= p(x_1)p(x_2 \mid x_1)...p(x_n \mid x_1, x_2, ..., x_{n-1}) \\
&= \prod_{i=1}^{n} p(x_i \mid x_1, x_2, ..., x_{i-1})
\end{aligned}
$$

  - x can be **any order / partition**
    - ✓ Reverse order is perfectly OK
    - ✓ Each x_i can be scalar / tensor

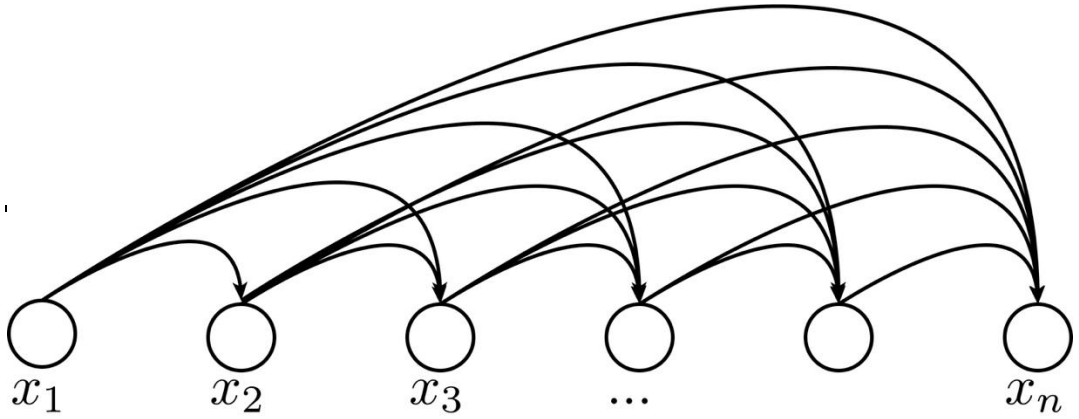


Introduction to Autoregressive Models

- **Autoregressive Models**
  - **Autoregression:** A way of modeling joint distribution by **a product of conditional** distributions

$$
\begin{aligned}
p(x_1, x_2, ..., x_n) &= p(x_1)p(x_2 \mid x_1)...p(x_n \mid x_1, x_2, ..., x_{n-1}) \\
&= \prod_{i=1}^{n} p(x_i \mid x_1, x_2, ..., x_{i-1})
\end{aligned}
$$

  - $p(\cdot|\cdot)$ can take **any form**
    - ✓ CPT, Neural Networks, trees, etc.
    - ✓ Discrete / Continuous variables



Introduction to Autoregressive Models

- **Autoregressive Models**
  - **Autoregression:** A way of modeling joint distribution by **a product of conditional** distributions

$$
\begin{aligned}
p(x_1, x_2, ..., x_n) &= p(x_1)p(x_2 \mid x_1)...p(x_n \mid x_1, x_2, ..., x_{n-1}) \\
&= \prod_{i=1}^{n} p(x_i \mid x_1, x_2, ..., x_{i-1})
\end{aligned}
$$

  - The factorization itself makes **no approximation** (simply chain-rule)
    - ✓ Decomposition is always valid for any variable order
      - ➔ Some are easier to model due to **inductive bias**

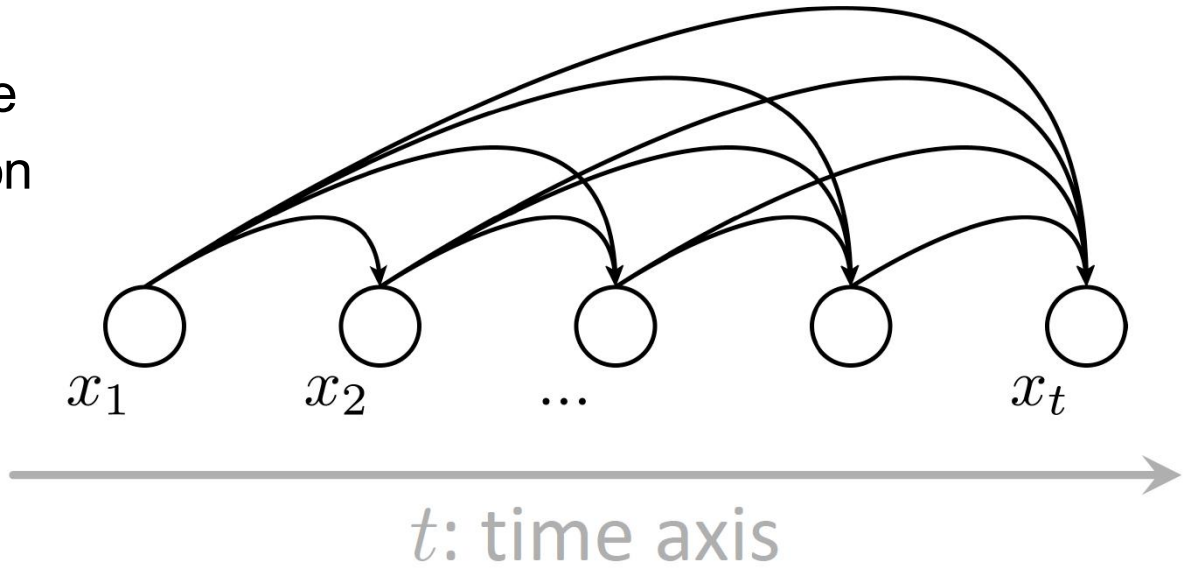
Introduction to Autoregressive Models

- **Inductive Bias (归纳偏执)**
  - **Assumptions built into a model that guide learning and generalization**
  - Some decomposition may **induce simple distributions**

Introduction to Autoregressive Models

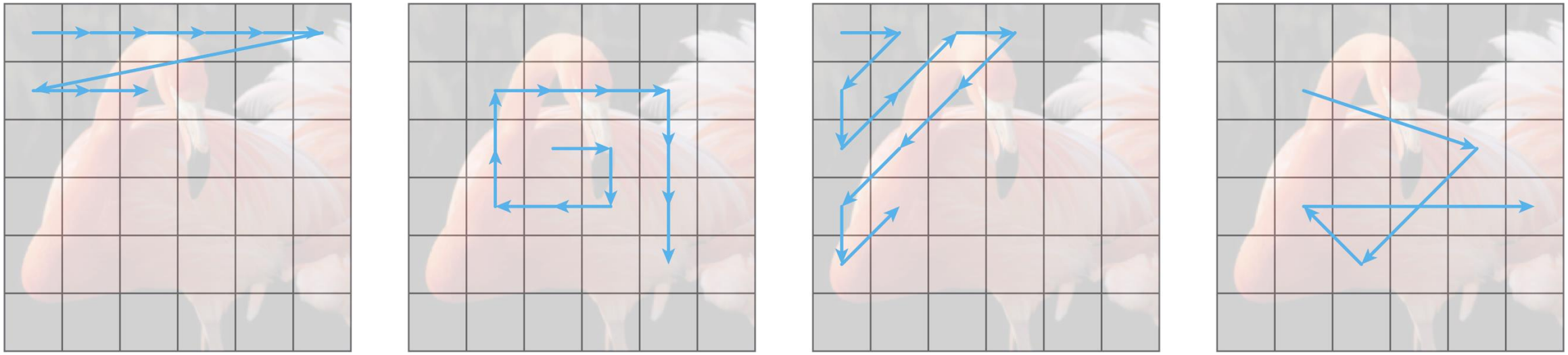
- **Inductive Bias**
  - ➢ Assumptions built into a model that guide learning and generalization
  - ➢ Some decomposition may **induce simple distributions**
    - ✓ **Temporal modeling** implies an inductive bias

Past observations provide context for future prediction



Introduction to Autoregressive Models

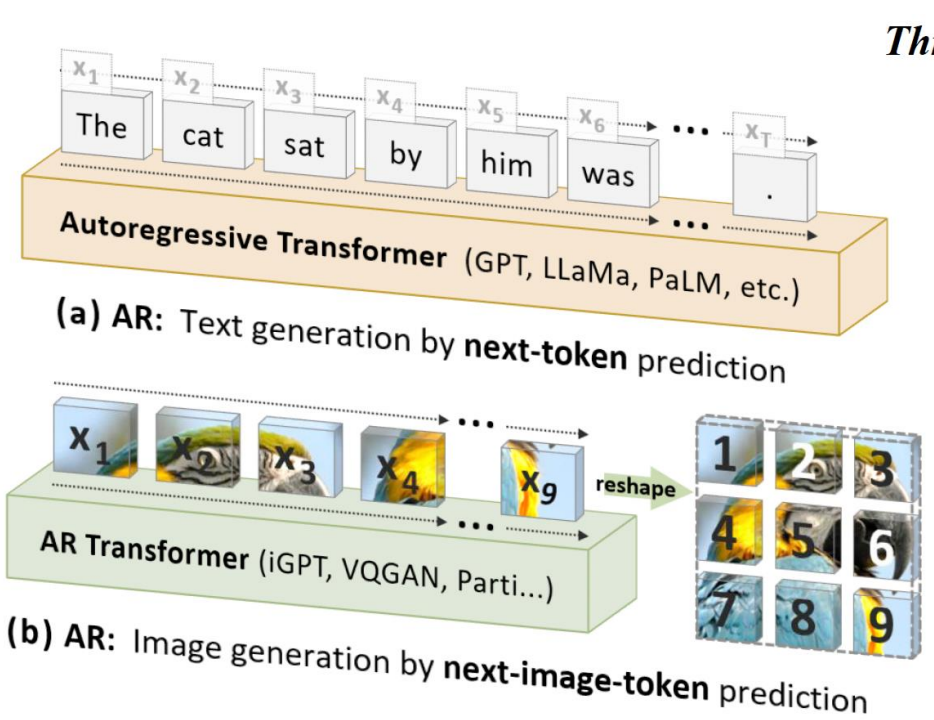
- **Inductive Bias**
  - ➢ Assumptions built into a model that guide learning and generalization
  - ➢ Some decomposition may **induce simple distributions**
    - ✓ **Spatial ordering** implies an inductive bias

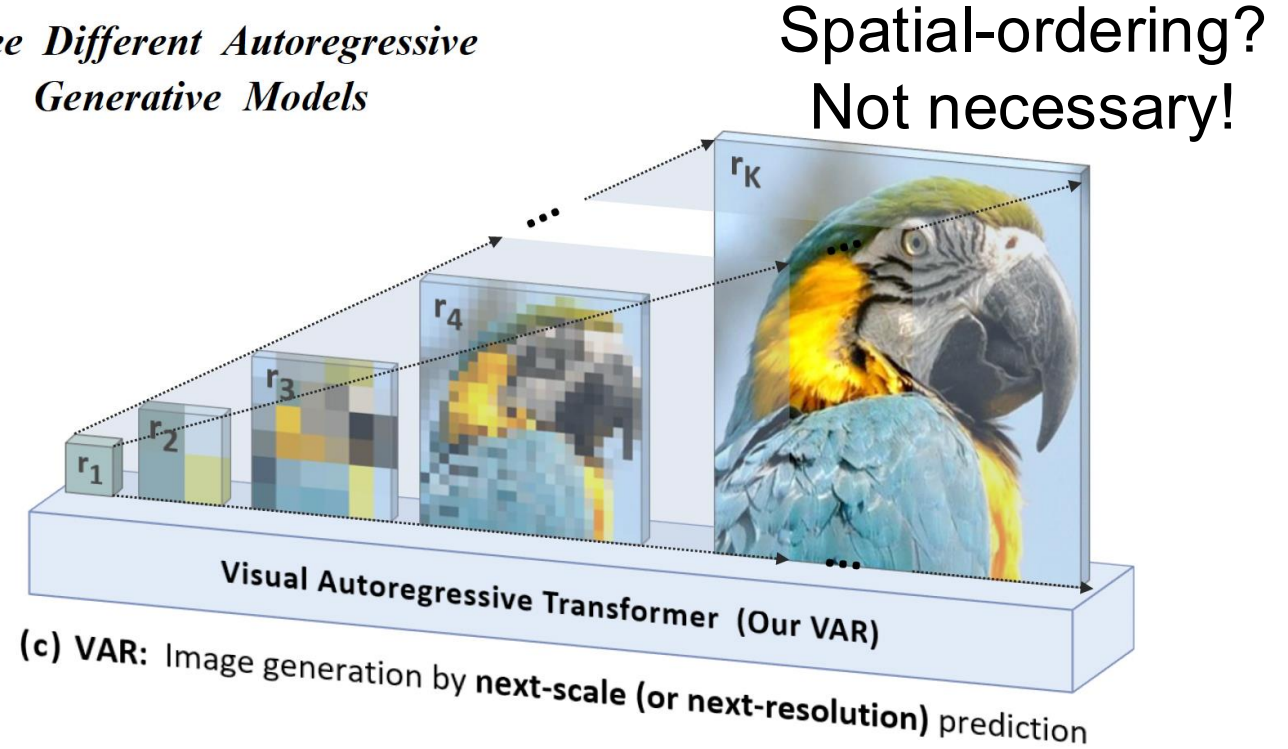


What is the inductive bias for each case?

Introduction to Autoregressive Models

- **Inductive Bias**
  - ➢ Assumptions built into a model that guide learning and generalization
  - ➢ Some decomposition may **induce simple distributions**





Introduction to Autoregressive Models

- **Inductive Bias**
  - Assumptions built into a model that guide learning and generalization
  - Some decomposition may **induce simple distributions**
  - Some decomposition may **allow similar parameterization**

$$p(x_1, x_2, ..., x_n) = p_\theta(x_1) p_\theta(x_2 \mid x_1) ... p_\theta(x_n \mid x_1, x_2, ..., x_{n-1})$$

- Conceptually, these are different mappings
- But we model them by **shared architectures** (which can be RNN, CNN, Transformer, ...)
- and by **shared weights** θ

Introduction to Autoregressive Models

- **Inductive Bias**
  - ➢ Assumptions built into a model that guide learning and generalization
  - ➢ Some decomposition may **induce simple distributions**
  - ➢ Some decomposition may **allow similar parameterization**

$$
\begin{aligned}
p(x_1, x_2, ..., x_n) &= p(x_1)p(x_2 \mid x_1)...p(x_n \mid x_1, x_2, ..., x_{n-1}) \\
&= \prod_{i=1}^{n} p(x_i \mid x_1, x_2, ..., x_{i-1})
\end{aligned}
$$

  - ➢ The decomposition makes no approximation / compromise

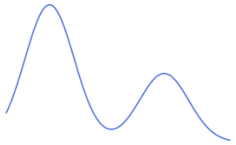
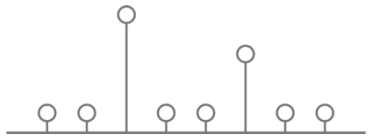
    ➔ But **inductive biases introduce approximations**

$$
p(x_1, x_2, ..., x_n) = p_\theta(x_1)p_\theta(x_2 \mid x_1)...p_\theta(x_n \mid x_1, x_2, ..., x_{n-1})
$$

Introduction to Autoregressive Models

- **Representing One Distribution**

$$p_\theta(x_i \mid x_1, x_2, \ldots, x_{i-1})$$

  - Network inputs: $x_1, x_2, \ldots, x_{i-1}$
  - Network output: a (conditional) distribution of x_i
    - ✓ Continuous 
    - ✓ Discrete (popular in AR models) 
  - With discrete distributions, this network **behaves like classification**
    - ✓ **'regression'** part of autoregression

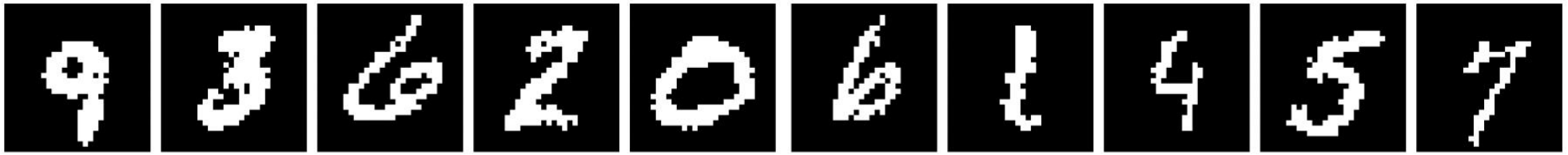


Lecture Outline

- Introduction to Autoregressive Models
- Basic Autoregressive Models
- Network Architectures for Autoregressive Modeling
- Inference and Training Scheme

Basic Autoregressive Models

- **A Toy Example on MNIST**
  - Consider a dataset $\mathcal{D}$ of (binarized) handwritten digits



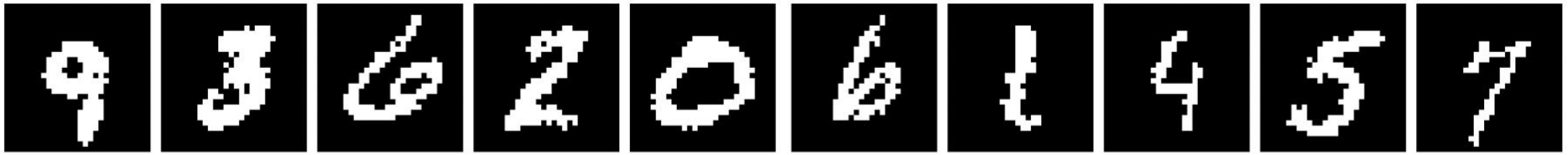
  - **Goal:** Learn a distribution $p(x) = p(x_1, \ldots, x_{784})$ over $x \in \{0,1\}^{28\times 28}$

    ➔ When $x_{new} \sim p(x)$, x_{new} looks like a handwritten digit

$$
\begin{aligned}
p(x_1, x_2, \ldots, x_n) &= p(x_1)p(x_2 \mid x_1)\ldots p(x_n \mid x_1, x_2, \ldots, x_{n-1}) \\
&= \prod_{i=1}^{n} p(x_i \mid x_1, x_2, \ldots, x_{i-1})
\end{aligned}
$$

Basic Autoregressive Models

- **A Toy Example on MNIST**
  - Consider a dataset $\mathcal{D}$ of (binarized) handwritten digits



  - **Goal:** Learn a distribution $p(x) = p(x_1, \ldots, x_{784})$ over $x \in \{0,1\}^{28\times 28}$

    ➔ When $x_{new} \sim p(x)$, x_{new} looks like a handwritten digit

  - **Solution:**

    ① Parameterize an autoregressive model family $\{p_\theta(\boldsymbol{x}), \theta \in \Theta\}$

    ② Solve optimal parameter θ^* on $\mathcal{D}$ with a certain objective $\mathcal{L}$

Recall

$$p(x_1, x_2, ..., x_n) = p(x_1)p(x_2 \mid x_1)...p(x_n \mid x_1, x_2, ..., x_{n-1})$$
$$= \prod_{i=1}^{n} p(x_i \mid x_1, x_2, ..., x_{i-1})$$

- **Neural Networks for Classification**
  - Discriminative models parameterize $p(Y = 1 \mid \boldsymbol{x})$, and assume that

$$p(Y = 1 \mid \boldsymbol{x}; \boldsymbol{\alpha}) = f(\boldsymbol{x}, \boldsymbol{\alpha})$$

① **Logistic regression:** Let $z(\boldsymbol{x}, \boldsymbol{\alpha}) = \alpha_0 + \sum \alpha_i x_i$ **Linear dependence**

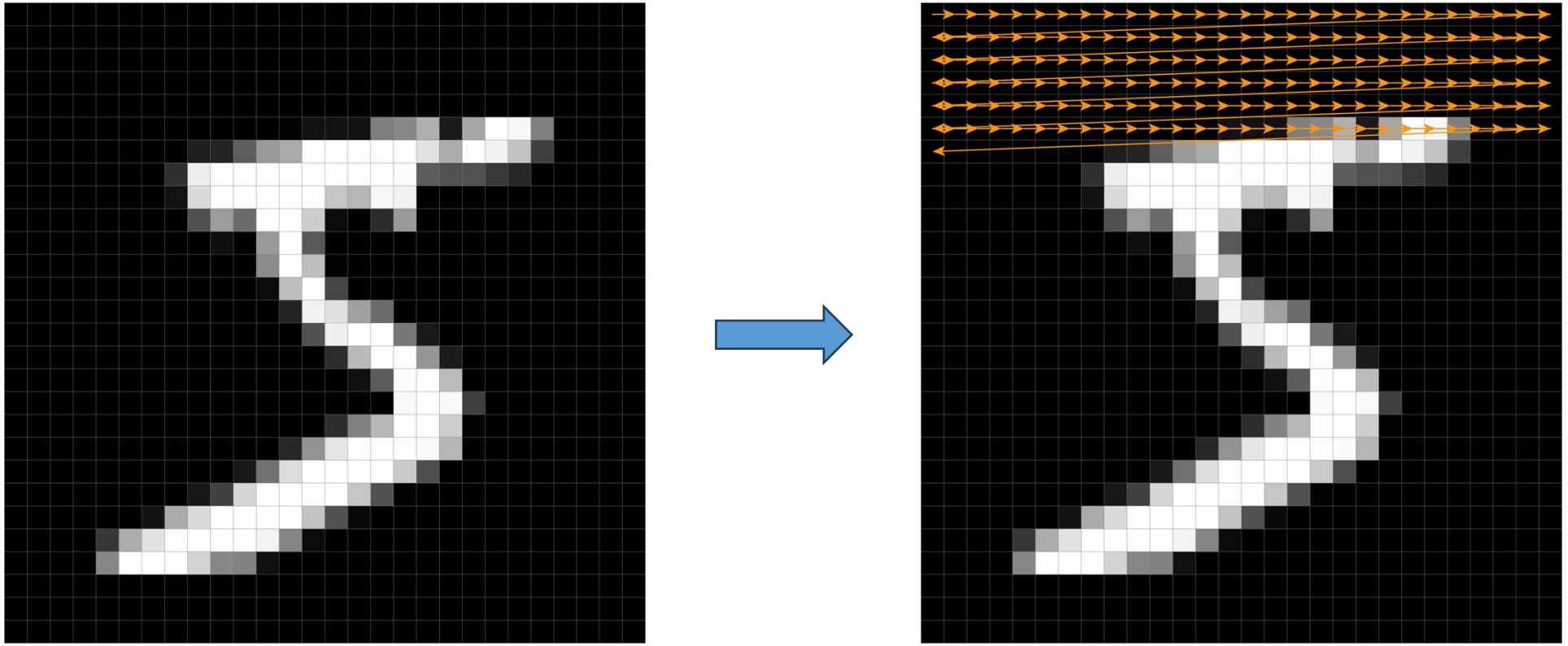
$$p_{logit}(Y = 1 \mid \boldsymbol{x}; \boldsymbol{\alpha}) = \sigma\big(z(\boldsymbol{x}, \boldsymbol{\alpha})\big), \ \ \sigma(z) = 1/(1 + e^{-z})$$

② **Multi-Layer Perceptron (MLP):** Let $h(\boldsymbol{A}, \boldsymbol{b}, \boldsymbol{x}) = g(\boldsymbol{A}\boldsymbol{x} + \boldsymbol{b})$ be a non-linear transformation of input $\boldsymbol{x}$

$$p_{MLP}(Y = 1 \mid \boldsymbol{x}; \boldsymbol{\alpha}, \boldsymbol{A}, \boldsymbol{b}) = \sigma\left(\alpha_0 + \sum_{i=1}^{h} \alpha_i \boldsymbol{h_i}\right)$$ **More flexible with more params.**

Basic Autoregressive Models

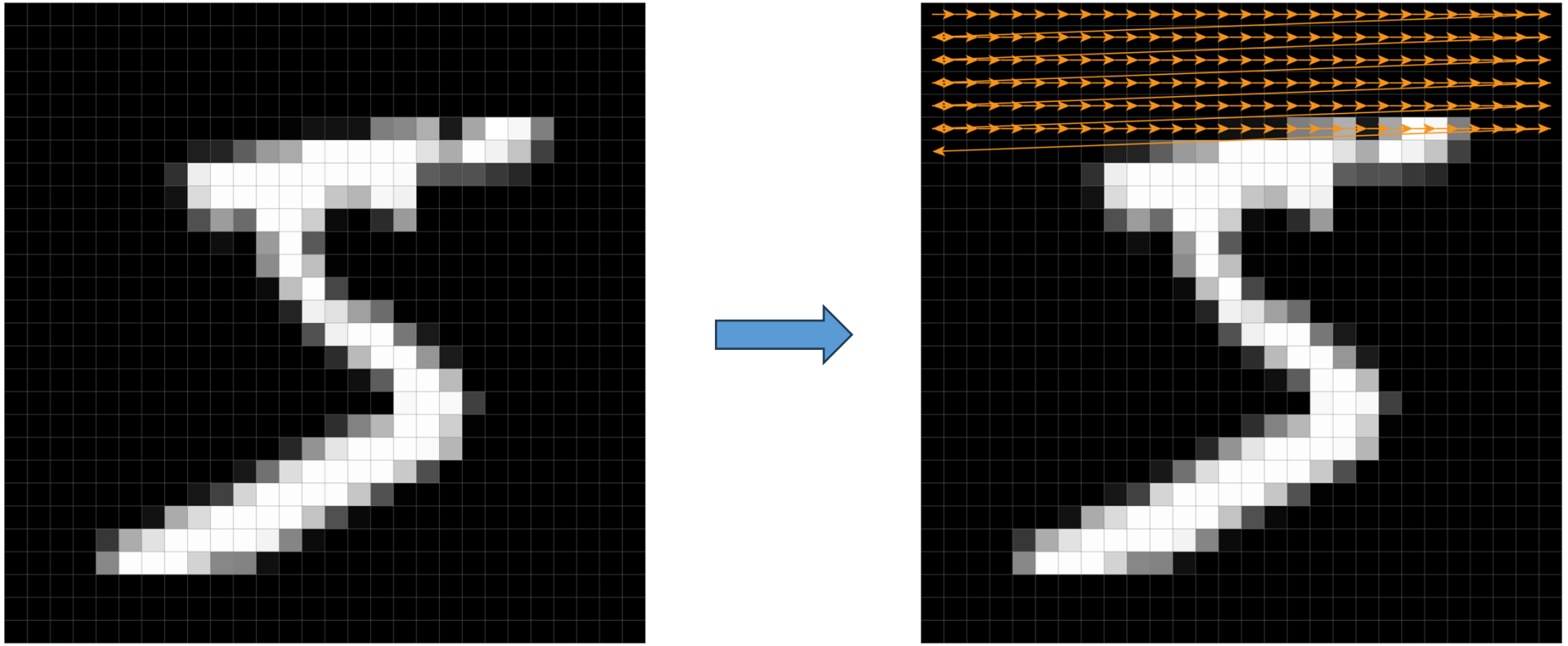
- **Logistic Regression-based Models**
  - Pick an ordering of the variables, e.g., a raster scan ordering



Raster scan ordering: from top-left X_1 to bottom-right X_{784}

Basic Autoregressive Models

- **Logistic Regression-based Models**
  - Write the probability factorization accordingly (i.e., Chain Rule)



$$p(x_1, \ldots, x_{784}) = p(x_1)\, p(\, x_2 \mid x_1\,)\, p(\, x_3 \mid x_1, x_2\,) \cdots p(\, x_n \mid x_1, \ldots, x_{n-1}\,)$$

Basic Autoregressive Models

- **Logistic Regression-based Models**

$$p(x_1, \dots, x_{784}) = p(x_1)\, p(\, x_2 \mid x_1\,)\, p(\, x_3 \mid x_1, x_2\,) \cdots p(\, x_n \mid x_1, \dots, x_{n-1}\,)$$

➔ $p(\, x_{784} \mid x_1, \dots, x_{783}\,)$ is too complex to represent in tabular form (CPT)

➢ **Solution: Parameterize** using logistic regression

$$p(x_1, \cdots, x_{784}) = p_{\text{CPT}}\left(x_1; \boldsymbol{\alpha}^{(1)}\right) p_{\text{logit}}\left(\, x_2 \mid x_1; \boldsymbol{\alpha}^{(2)}\,\right) \cdots p_{\text{logit}}\left(\, x_n \mid x_1, \cdots, x_{n-1}; \boldsymbol{\alpha}^{(n)}\,\right)$$

✓ $p_{\text{CPT}}(X_1 = 1; \boldsymbol{\alpha}^1) = \boldsymbol{\alpha}^1, \quad p(X_1 = 0) = 1 - \boldsymbol{\alpha}^1$

✓ $p_{\text{logit}}\left(\, X_2 = 1 \mid x_1; \boldsymbol{\alpha}^{(2)}\,\right) = \sigma\left(\boldsymbol{\alpha}_0^{(2)} + \boldsymbol{\alpha}_1^{(2)} x_1\right)$

✓ $p_{\text{logit}}\left(\, X_3 = 1 \mid x_1, x_2; \boldsymbol{\alpha}^{(2)}\,\right) = \sigma\left(\boldsymbol{\alpha}_0^{(3)} + \boldsymbol{\alpha}_1^{(3)} x_1 + \boldsymbol{\alpha}_2^{(3)} x_2\right)$

➢ **Autoregressive: predict the next variable given all the previous ones**

Basic Autoregressive Models

- **Logistic Regression-based Models**

$$p(x_1, \cdots, x_{784}) = p_{CPT}(x_1; \alpha^1)\, p_{\text{logit}}\left(x_2 \mid x_1; \boldsymbol{\alpha}^{(2)} \right) \cdots p_{\text{logit}}\left(x_n \mid x_1, \cdots, x_{n-1}; \boldsymbol{\alpha}^{(n)} \right)$$

  - Since $x_i \in \{0, 1\}$, $p_{\text{logit}}\left(x_i \mid x_1, \cdots, x_{i-1}; \boldsymbol{\alpha}^{(i)} \right)$ is a **conditional Bernoulli**
  - Let $\hat{x}$ denote **the parameter of the Bernoulli distribution**

$$\hat{x}_i = p_{\text{logit}}\left(X_i = 1 \mid x_{<i}; \boldsymbol{\alpha}^{(i)}\right) = \sigma\left(\alpha_0^i + \sum_{j=1}^{i-1} \alpha_j^i \cdot x_j \right)$$

    ① The probability that pixel X_i is on

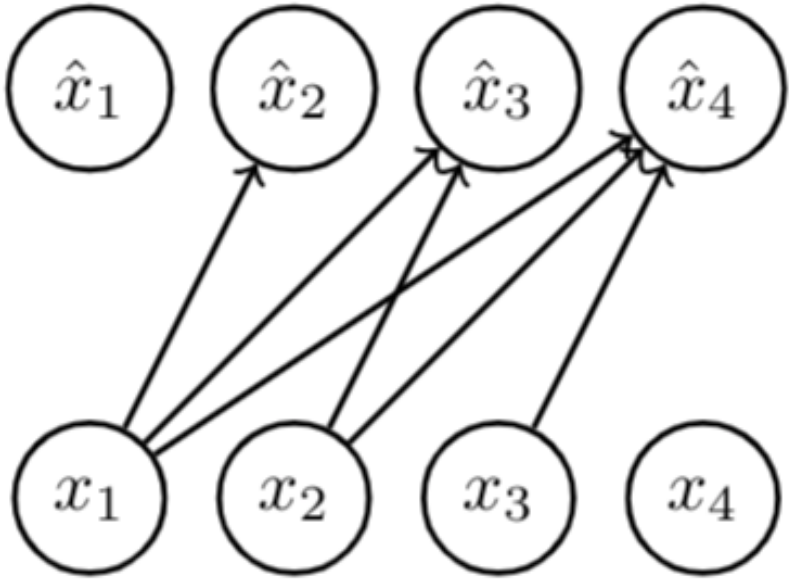
    ② A prediction of the next pixel X_i given the previous pixels $X_{<i}$

    ③ A reconstruction of X_i from partial input $X_{<i}$

Basic Autoregressive Models

- **Fully Visible Sigmoid Belief Network (FVSBN)**

Computational Graph of FVSBN

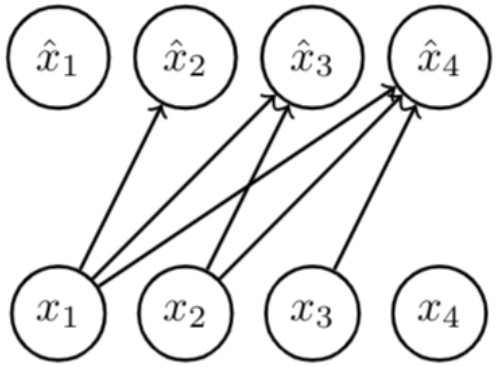


$$\hat{x}_i = p_{\text{logit}}\left(X_i = 1 \mid x_{<i}; \boldsymbol{\alpha}^{(i)}\right) = \sigma\left(\alpha_0^i + \sum_{j=1}^{i-1} \alpha_j^i \cdot x_j\right)$$

Neal, R. M., Connectionist Learning of Belief Networks. Artificial Intelligence, 56 (1), pp. 71-113, 1992.

Basic Autoregressive Models

- **Fully Visible Sigmoid Belief Network (FVSBN)**



➢ **How to evaluate $p(x_1, \ldots, x_{784})$?**

➔ Multiply all the conditionals in the autoregressive factorization

① Compute $\hat{x}_i = p_{\text{logit}}\left(x_i \mid x_{<i}; \boldsymbol{\alpha}^{(i)}\right)$ of each Bernoulli distribution

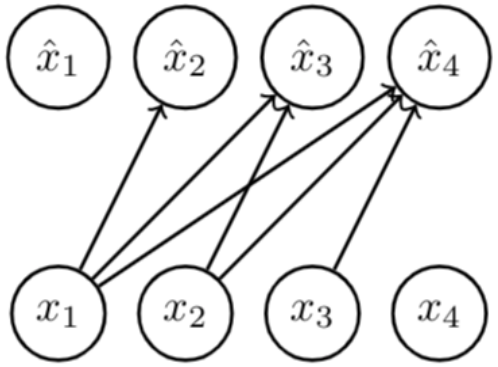
② Evaluate $p(x_1, \ldots, x_{784})$

$$p(X_1 = 0, X_2 = 1, X_3 = 1, X_4 = 0)$$

$$= (1 - \hat{x}_1) \times \hat{x}_2(X_1 = 0) \times \hat{x}_3(X_1 = 0, X_2 = 1) \times (1 - \hat{x}_4(X_1 = 0, X_2 = 1, X_3 = 1))$$

Basic Autoregressive Models

- **Fully Visible Sigmoid Belief Network (FVSBN)**



➢ **How to sample from $p(x_1, \ldots, x_{784})$?**

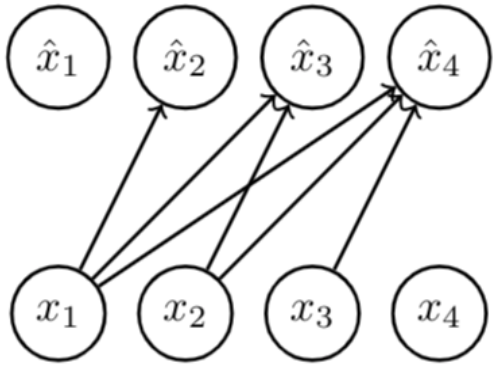
① Sample $\bar{x}_1 \sim p(x_1) = \mathrm{Ber}(\hat{x}_1)$

② Sample $\bar{x}_2 \sim p(\, x_2 \mid x_1 = \bar{x}_1\,) = p_{\mathrm{logit}}\big(\, x_2 \mid x_{<2}; \boldsymbol{\alpha}^{(2)}\,\big)$

③ Sample $\bar{x}_3 \sim p(\, x_3 \mid x_1 = \bar{x}_1, x_2 = \bar{x}_2\,) = p_{\mathrm{logit}}\big(\, x_3 \mid x_{<3}; \boldsymbol{\alpha}^{(3)}\,\big)$

Basic Autoregressive Models

- **Fully Visible Sigmoid Belief Network (FVSBN)**



  - **How to sample from $p(x_1, \ldots, x_{784})$?**

    ① Sample $\bar{x}_1 \sim p(x_1) = \mathrm{Ber}(\hat{x}_1)$

    ② Sample $\bar{x}_2 \sim p(x_2 \mid x_1 = \bar{x}_1) = p_{\mathrm{logit}}(x_2 \mid x_{<2}; \boldsymbol{\alpha}^{(2)})$

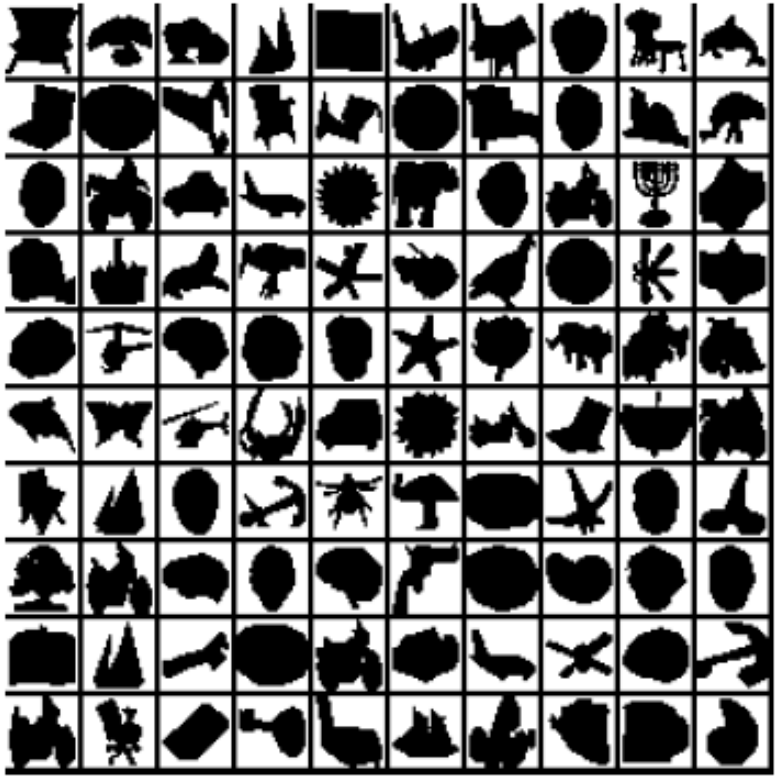
    ③ Sample $\bar{x}_3 \sim p(x_3 \mid x_1 = \bar{x}_1, x_2 = \bar{x}_2) = p_{\mathrm{logit}}(x_3 \mid x_{<3}; \boldsymbol{\alpha}^{(3)})$

  - **How many parameters?**

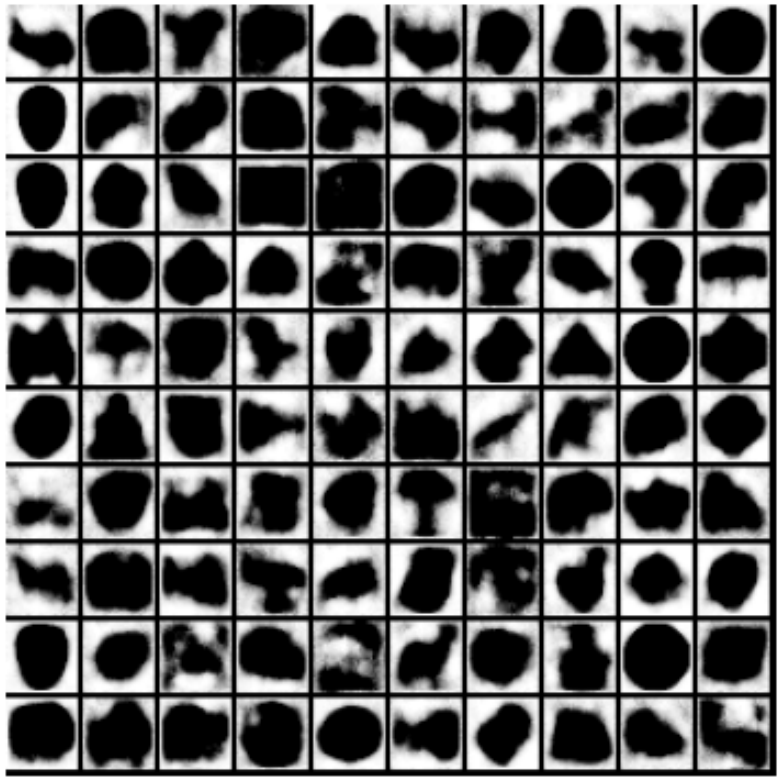
Basic Autoregressive Models

- **Fully Visible Sigmoid Belief Network (FVSBN)**

Caltech 101 Silhouettes



Training Data



Sampling Results

Learning Deep Sigmoid Belief Networks with Data Augmentation, Gan et al., PMLR, 2015.

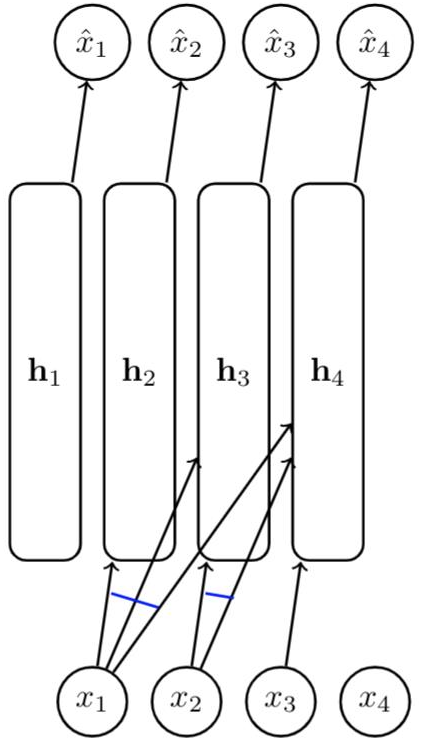
Basic Autoregressive Models

- **From Logistic Regression to MLP**
  - **How to improve the modeling of conditionals?**
    - ➔ Replace logistic regression with a three-layer MLP
  - We now compute $\hat{x}_i$ using a neural network

$$\mathbf{h}_i = \sigma(A_i \mathbf{x}_{<i} + \mathbf{c}_i)$$

$$\hat{x}_i = p(x_i | x_1, \cdots, x_{i-1}; \underbrace{A_i, \mathbf{c}_i, \boldsymbol{\alpha}_i, b_i}_{\text{parameters}}) = \sigma(\boldsymbol{\alpha}_i \mathbf{h}_i + b_i)$$

$$\mathbf{h}_2 = \sigma\left(\underbrace{\begin{pmatrix}\vdots\end{pmatrix}}_{A_2} x_1 + \underbrace{\begin{pmatrix}\vdots\end{pmatrix}}_{c_2}\right) \quad \mathbf{h}_3 = \sigma\left(\underbrace{\begin{pmatrix}\vdots & \vdots\end{pmatrix}}_{A_3} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} + \underbrace{\begin{pmatrix}\vdots\end{pmatrix}}_{c_3}\right)$$



NADE

Computational Complexity?

Basic Autoregressive Models

- **From Logistic Regression to MLP**
  - ➢ **How to reduce the number of parameters?**
    - ➔ **Share weights** across dimensions

$$\mathbf{h}_i = \sigma(A_i \mathbf{x}_{<i} + \mathbf{c}_i)$$

$$\hat{x}_i = p(x_i | x_1, \cdots, x_{i-1}; \underbrace{A_i, \mathbf{c}_i, \alpha_i, b_i}_{\text{parameters}}) = \sigma(\alpha_i \mathbf{h}_i + b_i)$$

Shared weights $W_{<i}$ and constant c

$$\mathbf{h}_i = \sigma(W_{\cdot,<i} \mathbf{x}_{<i} + \mathbf{c})$$

$$\hat{x}_i = p(x_i | x_1, \cdots, x_{i-1}) = \sigma(\alpha_i \mathbf{h}_i + b_i)$$

NO sharing for hidden → output

Basic Autoregressive Models

- **From Logistic Regression to MLP**
  - ➢ **How to reduce the number of parameters?**
    - ➔ **Share weights** across dimensions

$$\mathbf{h}_i = \sigma(A_i \mathbf{x}_{<i} + \mathbf{c}_i)$$
$$\hat{x}_i = p(x_i | x_1, \cdots, x_{i-1}; \underbrace{A_i, \mathbf{c}_i, \alpha_i, b_i}_{\text{parameters}}) = \sigma(\alpha_i \mathbf{h}_i + b_i)$$

Shared weights $W_{<i}$ and constant c

$$\mathbf{h}_i = \sigma(W_{\cdot,<i} \mathbf{x}_{<i} + \mathbf{c})$$
$$\hat{x}_i = p(x_i | x_1, \cdots, x_{i-1}) = \sigma(\alpha_i \mathbf{h}_i + b_i)$$

NO sharing for hidden → output

$$\mathbf{h}_2 = \sigma\left(\underbrace{\begin{pmatrix} \vdots \end{pmatrix}}_{A_2} x_1 + \underbrace{\begin{pmatrix} \vdots \end{pmatrix}}_{c_2}\right) \quad \mathbf{h}_3 = \sigma\left(\underbrace{\begin{pmatrix} \vdots & \vdots \end{pmatrix}}_{A_3} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} + \underbrace{\begin{pmatrix} \vdots \end{pmatrix}}_{c_3}\right)$$

$$\mathbf{h}_2 = \sigma\left(\underbrace{\begin{pmatrix} \vdots \\ w_1 \\ \vdots \end{pmatrix}}_{W_{\cdot,<2}} x_1 + \mathbf{c}\right) \quad \mathbf{h}_3 = \sigma\left(\underbrace{\begin{pmatrix} \vdots & \vdots \\ w_1 & w_2 \\ \vdots & \vdots \end{pmatrix}}_{W_{\cdot,<3}} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}\right) \quad \mathbf{h}_4 = \sigma\left(\underbrace{\begin{pmatrix} \vdots & \vdots & \vdots \\ w_1 & w_2 & w_3 \\ \vdots & \vdots & \vdots \end{pmatrix}}_{W_{\cdot,<4}} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}\right)$$

Computational Complexity?

Basic Autoregressive Models

- **From Logistic Regression to MLP**
  - ➢ **How to reduce the number of parameters?**
    - ➔ **Share weights** across dimensions

$$\mathbf{h}_i = \sigma(A_i \mathbf{x}_{<i} + \mathbf{c}_i)$$
$$\hat{x}_i = p(x_i | x_1, \cdots, x_{i-1}; \underbrace{A_i, \mathbf{c}_i, \alpha_i, b_i}_{\text{parameters}}) = \sigma(\alpha_i \mathbf{h}_i + b_i)$$

Shared weights $W_{<i}$ and constant c

$$\mathbf{h}_i = \sigma(W_{\cdot,<i} \mathbf{x}_{<i} + \mathbf{c})$$
$$\hat{x}_i = p(x_i | x_1, \cdots, x_{i-1}) = \sigma(\alpha_i \mathbf{h}_i + b_i)$$

NO sharing for hidden → output

$$\mathbf{h}_2 = \sigma\left(\underbrace{\begin{pmatrix} \vdots \end{pmatrix}}_{A_2} x_1 + \underbrace{\begin{pmatrix} \vdots \end{pmatrix}}_{c_2}\right) \quad \mathbf{h}_3 = \sigma\left(\underbrace{\begin{pmatrix} \vdots & \vdots \end{pmatrix}}_{A_3} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} + \underbrace{\begin{pmatrix} \vdots \end{pmatrix}}_{c_3}\right)$$

$$\mathbf{h}_2 = \sigma\left(\underbrace{\begin{pmatrix} \vdots \\ w_1 \\ \vdots \end{pmatrix}}_{W_{\cdot,<2}} x_1 + \mathbf{c}\right) \quad \mathbf{h}_3 = \sigma\left(\underbrace{\begin{pmatrix} \vdots & \vdots \\ w_1 & w_2 \\ \vdots & \vdots \end{pmatrix}}_{W_{\cdot,<3}} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}\right) \quad \mathbf{h}_4 = \sigma\left(\underbrace{\begin{pmatrix} \vdots & \vdots & \vdots \\ w_1 & w_2 & w_3 \\ \vdots & \vdots & \vdots \end{pmatrix}}_{W_{\cdot,<4}} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}\right)$$

How to generalize to grayscale?

Basic Autoregressive Models

- **From Logistic Regression to MLP**
  - ➢ **How to reduce the number of parameters?**
    - ➔ **Share weights** across dimensions

Shared weights $W_{<i}$ and constant c

$$\mathbf{h}_i = \sigma(A_i \mathbf{x}_{<i} + \mathbf{c}_i)$$

$$\hat{x}_i = p(x_i | x_1, \cdots, x_{i-1}; \underbrace{A_i, \mathbf{c}_i, \boldsymbol{\alpha}_i, b_i}_{\text{parameters}}) = \sigma(\boldsymbol{\alpha}_i \mathbf{h}_i + b_i)$$

$$\Rightarrow$$

$$\mathbf{h}_i = \sigma(W_{\cdot,<i} \mathbf{x}_{<i} + \mathbf{c})$$

$$\hat{x}_i = p(x_i | x_1, \cdots, x_{i-1}) = \sigma(\boldsymbol{\alpha}_i \mathbf{h}_i + b_i)$$

NO sharing for hidden → output

$$\mathbf{h}_2 = \sigma\left(\underbrace{\begin{pmatrix}\vdots\\ \vdots\end{pmatrix}}_{A_2} x_1 + \underbrace{\begin{pmatrix}\vdots\\ \vdots\end{pmatrix}}_{c_2}\right) \quad \mathbf{h}_3 = \sigma\left(\underbrace{\begin{pmatrix}\vdots & \vdots\\ \vdots & \vdots\end{pmatrix}}_{A_3} \begin{pmatrix}x_1\\ x_2\end{pmatrix} + \underbrace{\begin{pmatrix}\vdots\\ \vdots\end{pmatrix}}_{c_3}\right)$$

$$\Rightarrow \quad \mathbf{h}_2 = \sigma\left(\underbrace{\begin{pmatrix}\vdots\\ \boldsymbol{w}_1\\ \vdots\end{pmatrix}}_{W_{\cdot,<2}} x_1 + \mathbf{c}\right) \quad \mathbf{h}_3 = \sigma\left(\underbrace{\begin{pmatrix}\vdots & \vdots\\ \boldsymbol{w}_1 & \boldsymbol{w}_2\\ \vdots & \vdots\end{pmatrix}}_{W_{\cdot,<3}} \begin{pmatrix}x_1\\ x_2\end{pmatrix}\right) \quad \mathbf{h}_4 = \sigma\left(\underbrace{\begin{pmatrix}\vdots & \vdots & \vdots\\ \boldsymbol{w}_1 & \boldsymbol{w}_2 & \boldsymbol{w}_3\\ \vdots & \vdots & \vdots\end{pmatrix}}_{W_{\cdot,<4}} \begin{pmatrix}x_1\\ x_2\\ x_3\end{pmatrix}\right)$$

How to compute more efficiently?

Basic Autoregressive Models

- **Neural Autoregressive Density Estimation (NADE)**

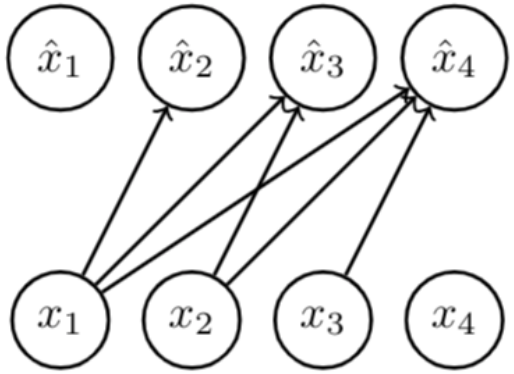


Sampling Results Conditional prob. $\hat{x}_i$

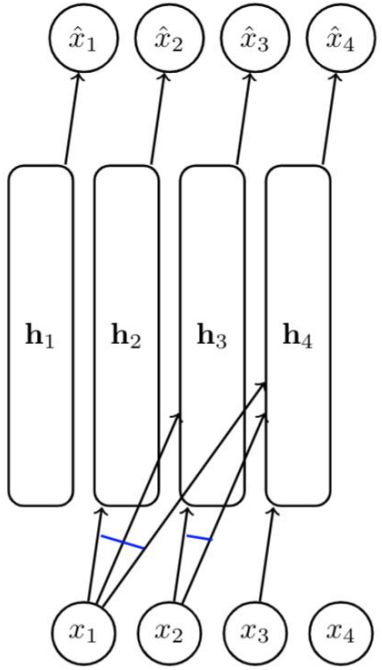
The Neural Autoregressive Distribution Estimator, Hugo Larochelle, Iain Murray, PMLR, 2011.

Basic Autoregressive Models

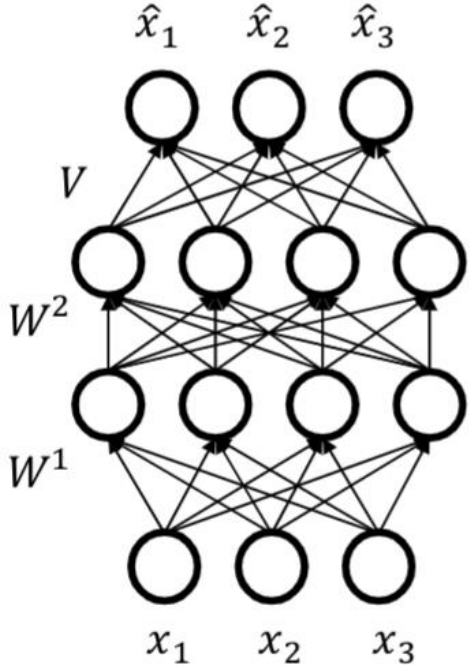
- **Autoregressive Models vs. Auto-encoders**



FVSBN



NADE



Autoencoder

➢ **Note:** Autoregressive models look very **similar to an autoencoder**

Basic Autoregressive Models

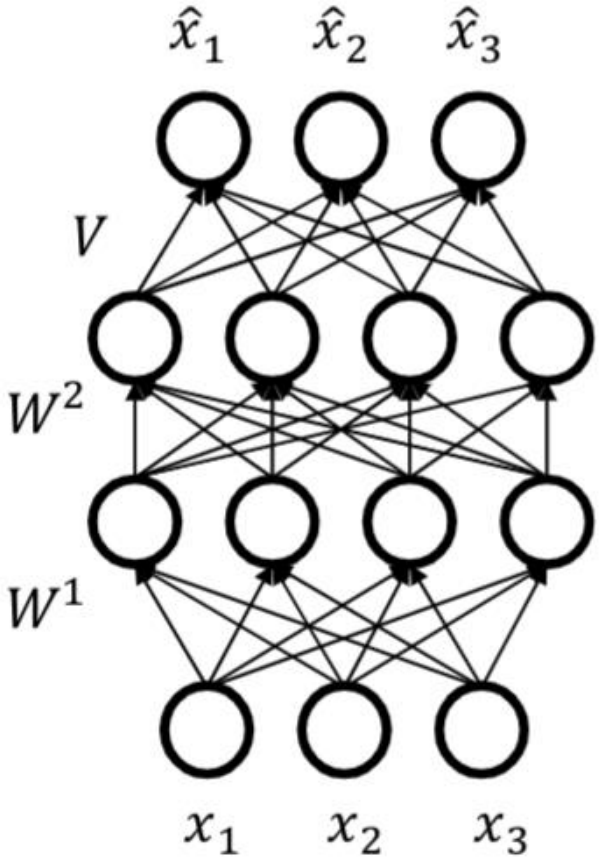
- **Autoregressive Models vs. Auto-encoders**
  - An autoencoder consists:
    - ✓ Encoder $e(\cdot)$

$$e(x) = \sigma(W^2(W^1 x + b^1) + b^2)$$

    - ✓ Decode $d(\cdot)$

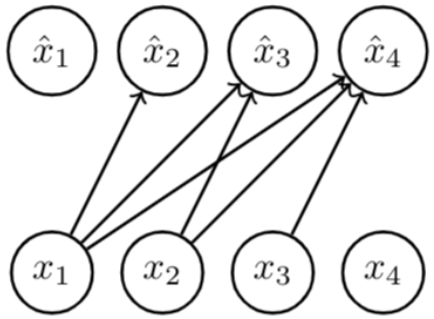
$$d(h) = \sigma(Vh + c)$$

    - ✓ Objective
      - $d\big(e(\boldsymbol{x})\big) \approx \boldsymbol{x}$ (i.e, input reconstruction)
      - $e(\boldsymbol{x})$ is restricted to learn meaningful representations of $\boldsymbol{x}$
  - A vanilla Autoencoder is not a generative model

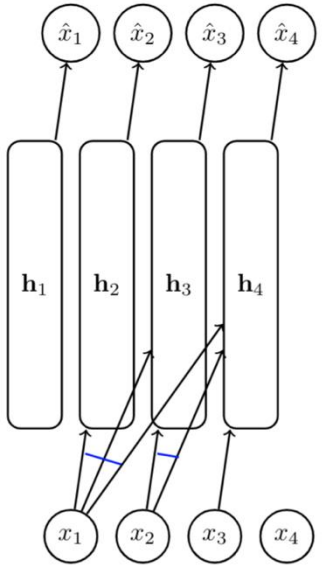


Basic Autoregressive Models

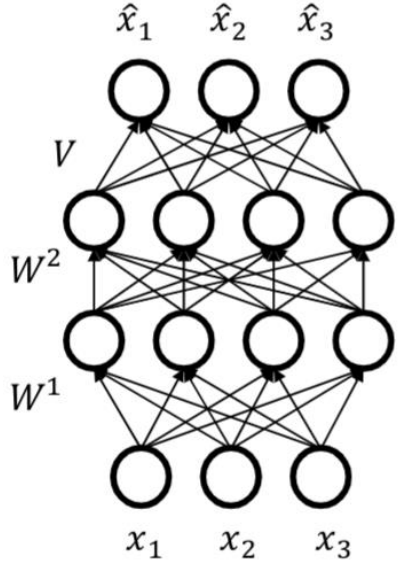
- **Autoregressive Models vs. Auto-encoders**



FVSBN



NADE

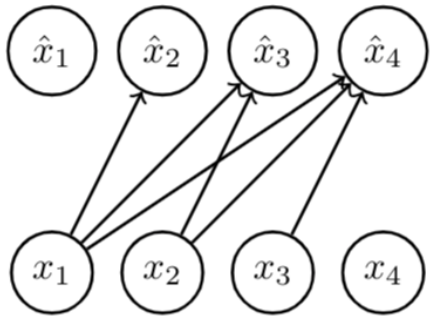


Autoencoder

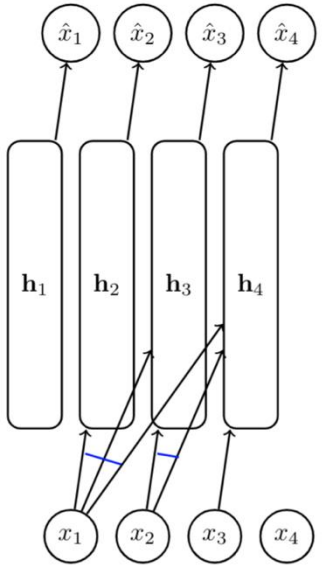
- What are the principal differences between AR models and Autoencoder, on the surface?
  - ✓ Or say, in terms of the computation graph / data flow?

Basic Autoregressive Models

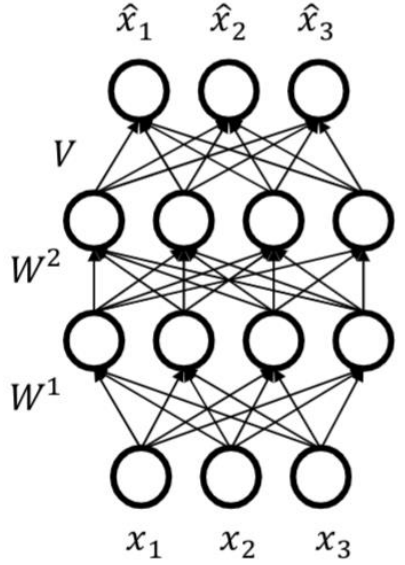
- **Autoregressive Models vs. Auto-encoders**



FVSBN



NADE

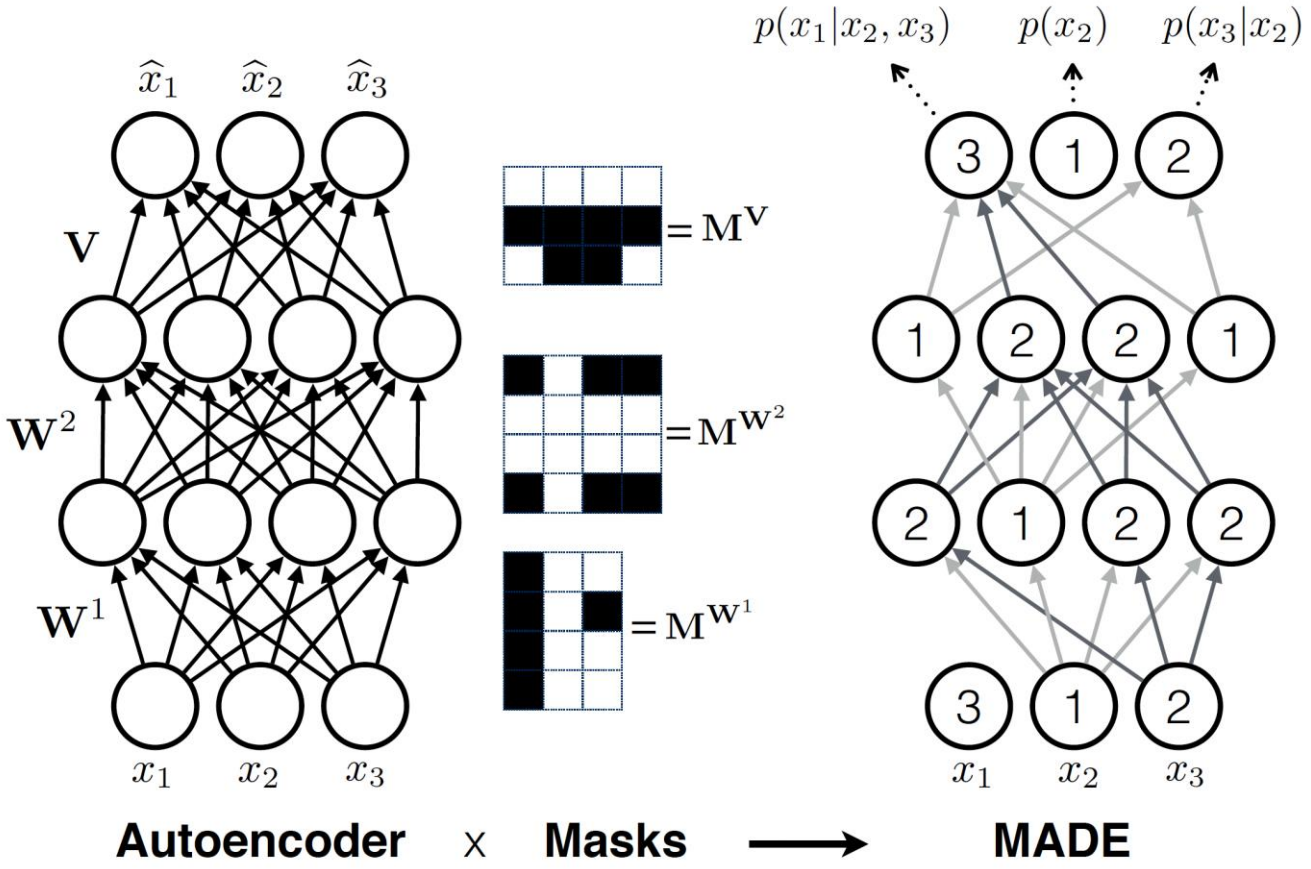


Autoencoder

- An Autoregressive model ➔ An autoencoder that **predicts a shifted input**
  - ✓ $\hat{x}_i$ only depends on its **ancestors** $x_{<i}$
  - ✓ Computational graph: **an autoencoder with masked edges** (weights)

Basic Autoregressive Models

- **Masked Auto-encoders**
  - Disallow paths (zero weights) in an autoencoder to make it autoregressive



$$p(x_1, x_2, x_3)$$
$$= p(x_2)\, p(x_3 \mid x_2)\, p(x_1 \mid x_2, x_3)$$

MADE: Masked Autoencoder for Distribution Estimation, Germain et al. ICML, 2015.



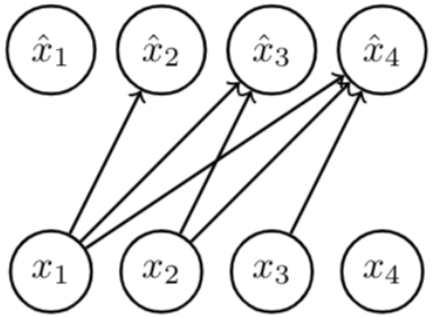
Lecture Outline

- Introduction to Autoregressive Models
- Basic Autoregressive Models
- Network Architectures for Autoregressive Modeling
- Inference and Training Scheme

Network Architectures for Autoregressive Modeling

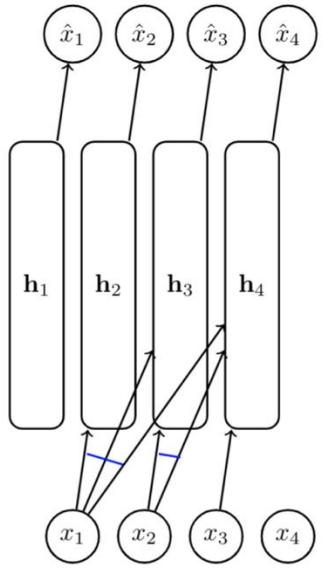
- **Framework Comparison**

FVSBN



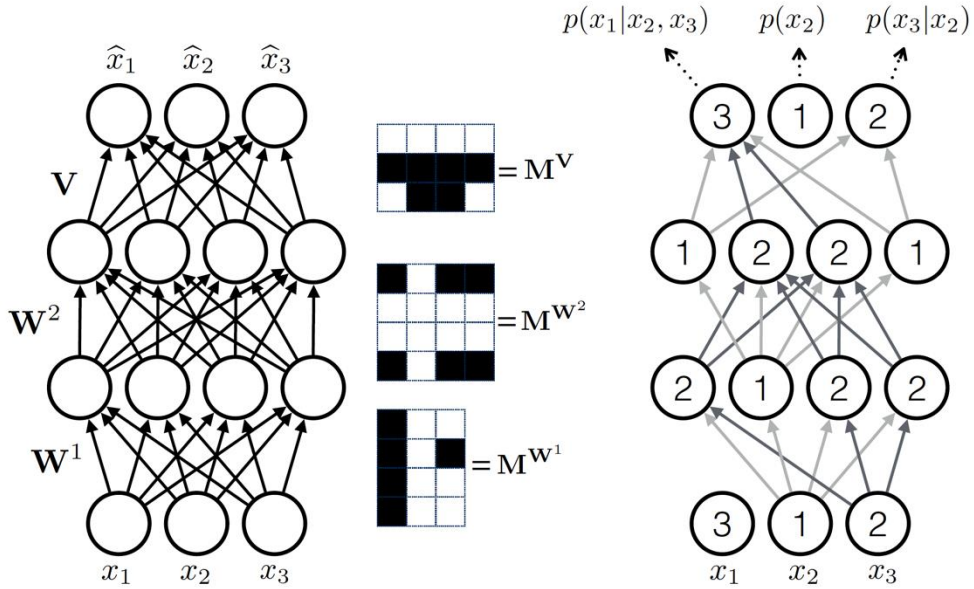
Logistic Regression

NADE



Neural Network & Weight Sharing

MADE

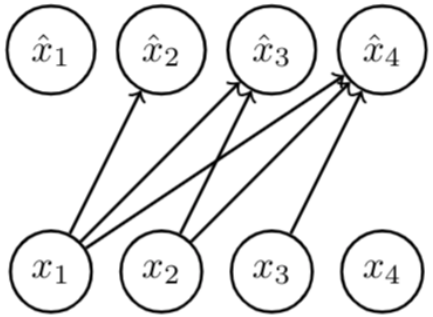


Autoencoder + Weight Masking

Network Architectures for Autoregressive Modeling

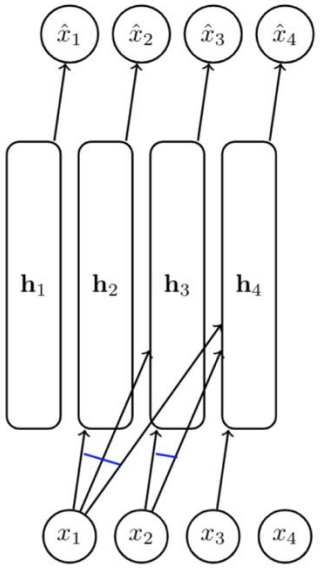
- **Framework Comparison**

FVSBN



Logistic Regression

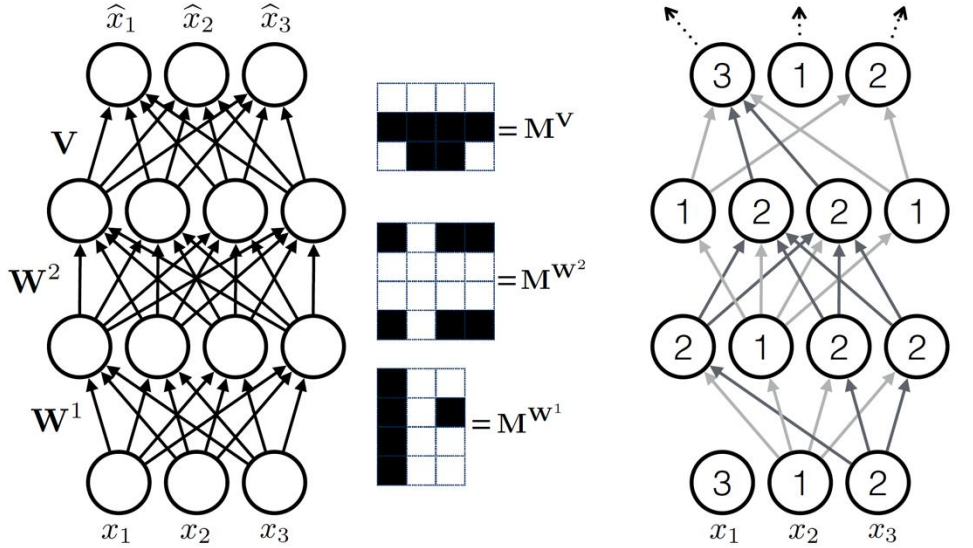
NADE



Neural Network w\o Weight Sharing

Multiple networks, varying inputs

MADE

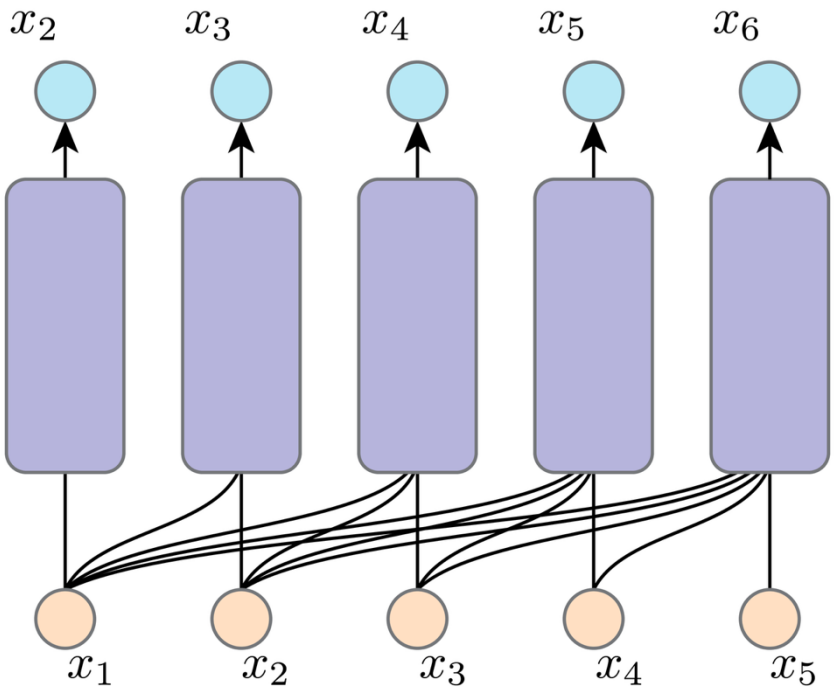


Autoencoder + Weight Masking

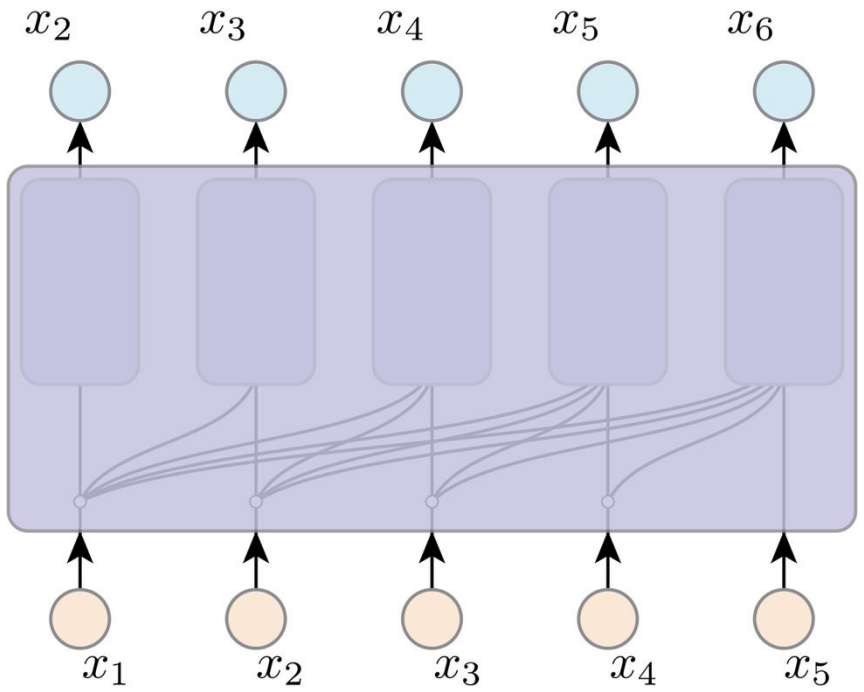
One network w/ causal condition

Network Architectures for Autoregressive Modeling

- **Framework Comparison**



- Multiple networks
- Varying number of inputs

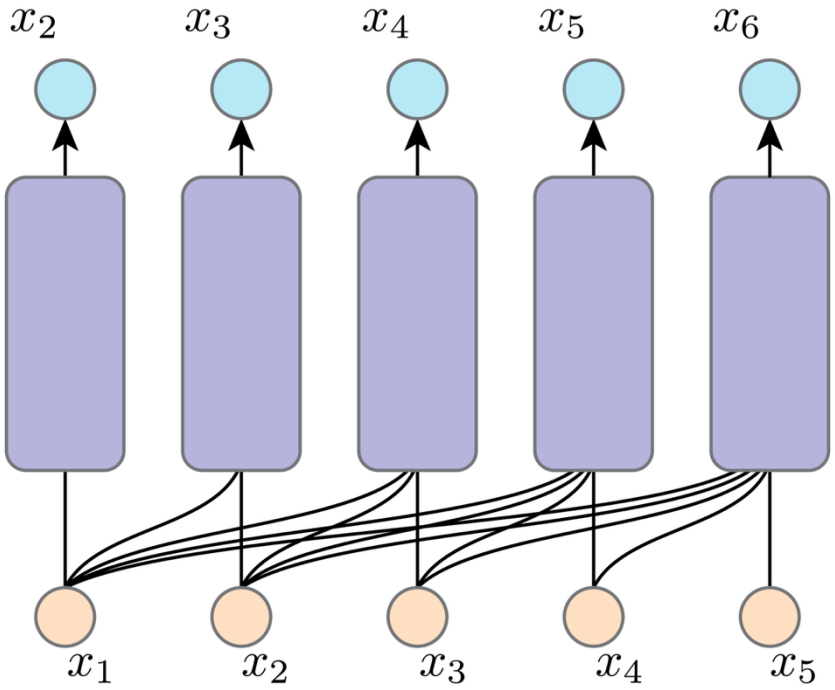


- One single network
- Receive all input at once
- **Output x_i only depends on $x_{<i}$**

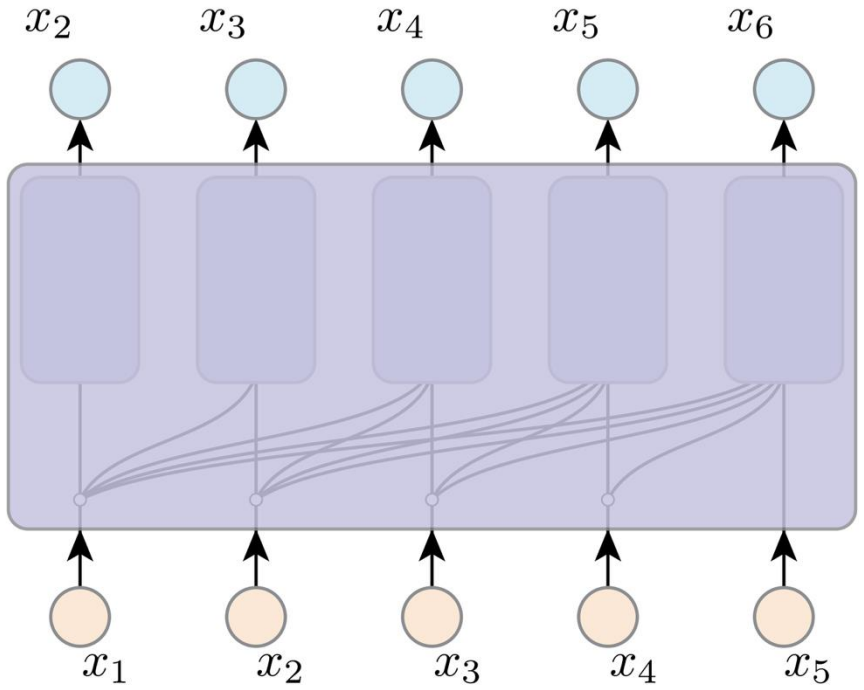
Network Architectures for Autoregressive Modeling

- **Framework Comparison**

$$p(x_1, x_2, \ldots, x_n) = \prod_{i=1}^{n} p(x_i \mid x_1, x_2, \ldots, x_{i-1})$$



- Multiple networks
- Varying number of inputs
- AR-ness is **Structural (hard)**



- One single network
- Receive all input at once
- AR-ness is **Imposed (soft)**

Network Architectures for Autoregressive Modeling

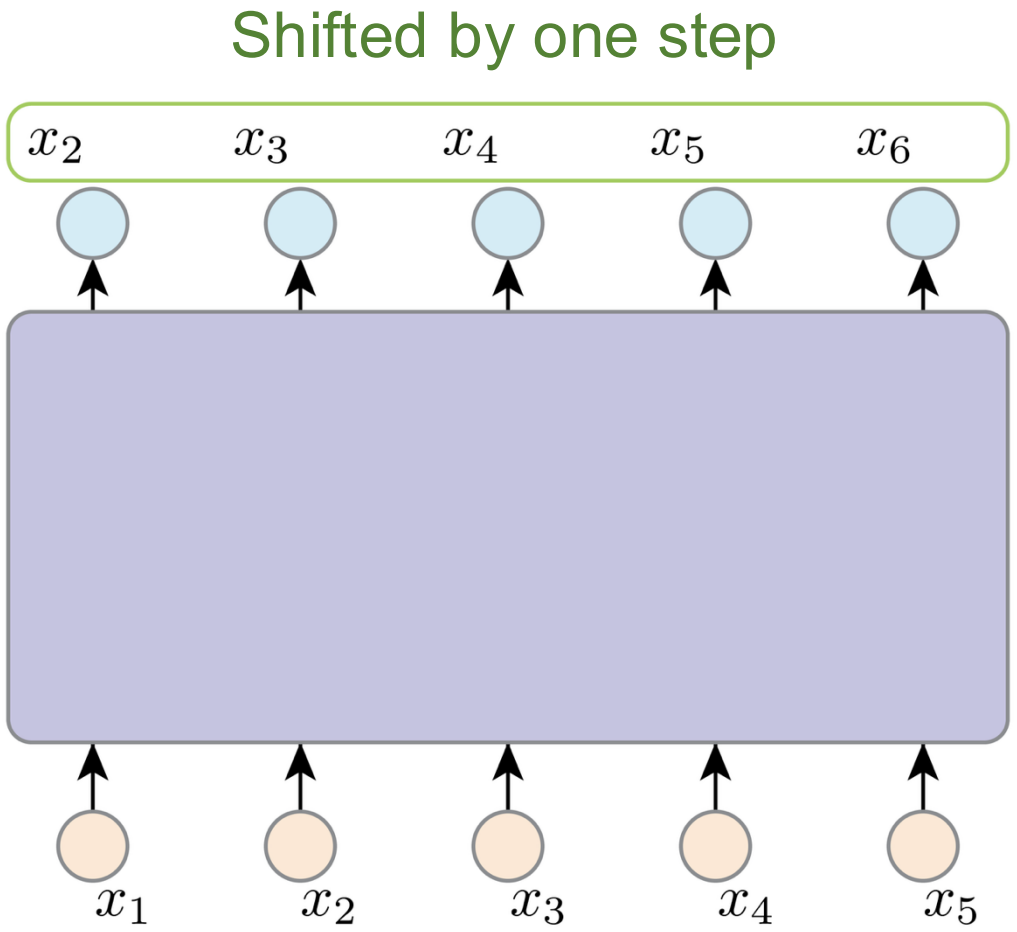
- **Common Architectures**
  - ➢ We can implement:

$$p(x_1, x_2, \dots, x_n) = \prod_{i=1}^{n} p(x_i \mid x_1, x_2, \dots, x_{i-1})$$

by a single network, with

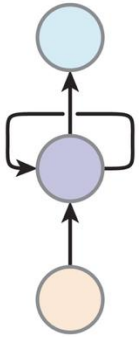
- ✓ Shared architecture
- ✓ Shared weights
- ✓ Shared computation

  - ➢ If: **Output x_i only depends on $x_{<i}$**

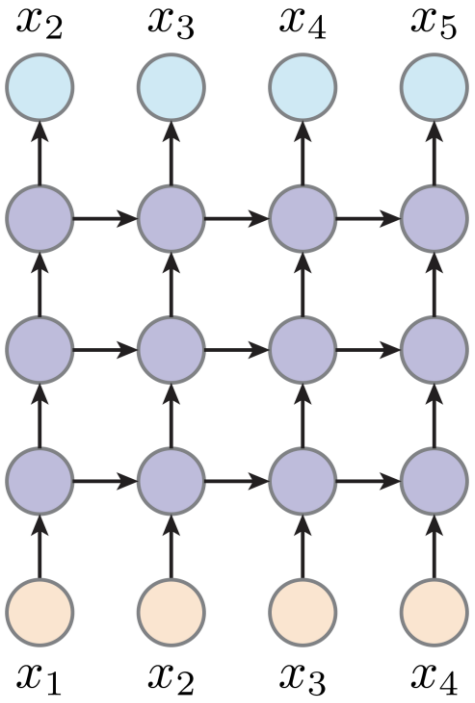


Network Architectures for Autoregressive Modeling

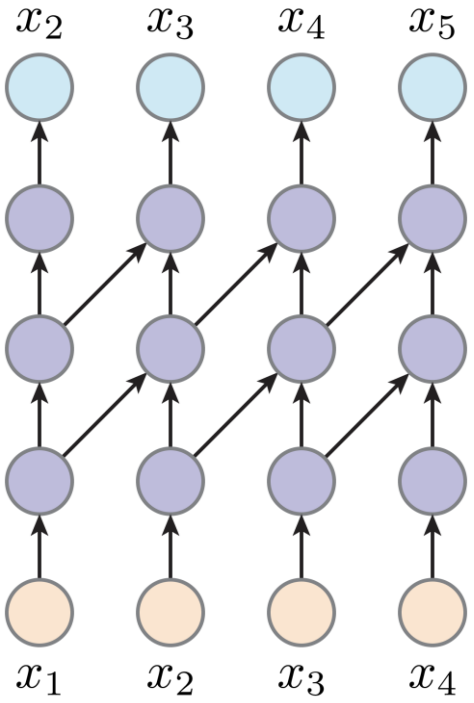
- **Common Architectures**



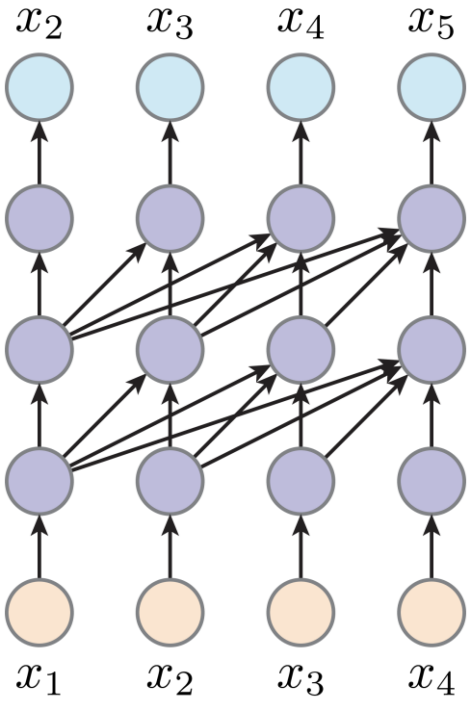
RNN Unit (folded)



RNN Unit (unfolded)



CNN

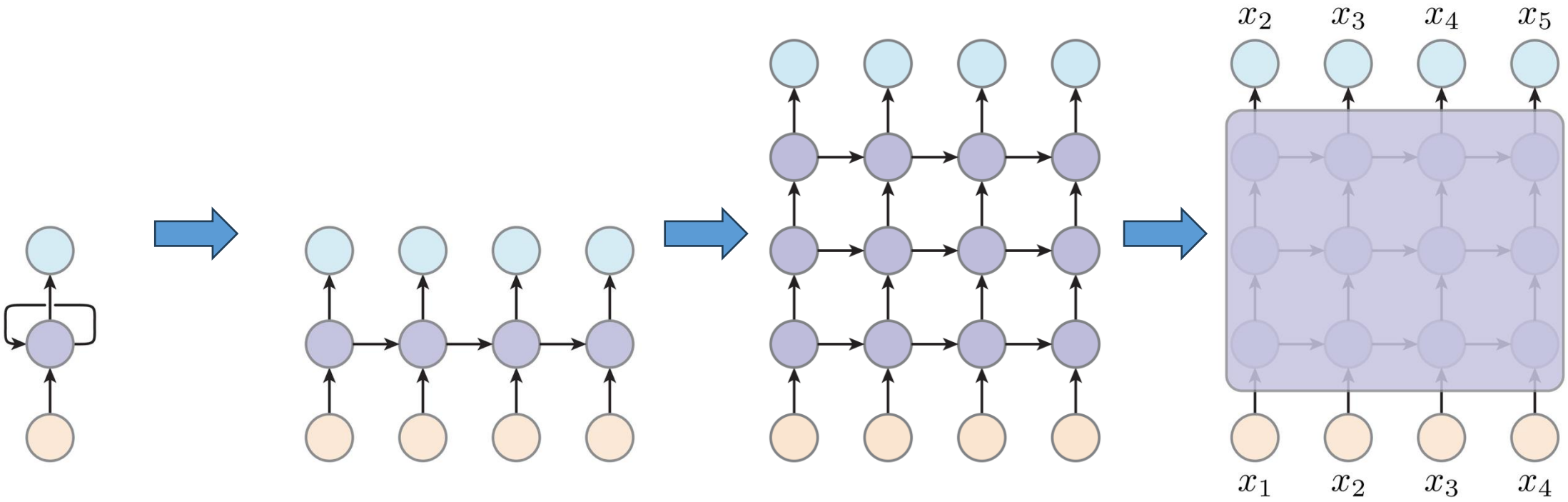


Attention

Network Architectures for Autoregressive Modeling

- **Common Architectures: RNN**

Shifted by one step



RNN Unit (folded)

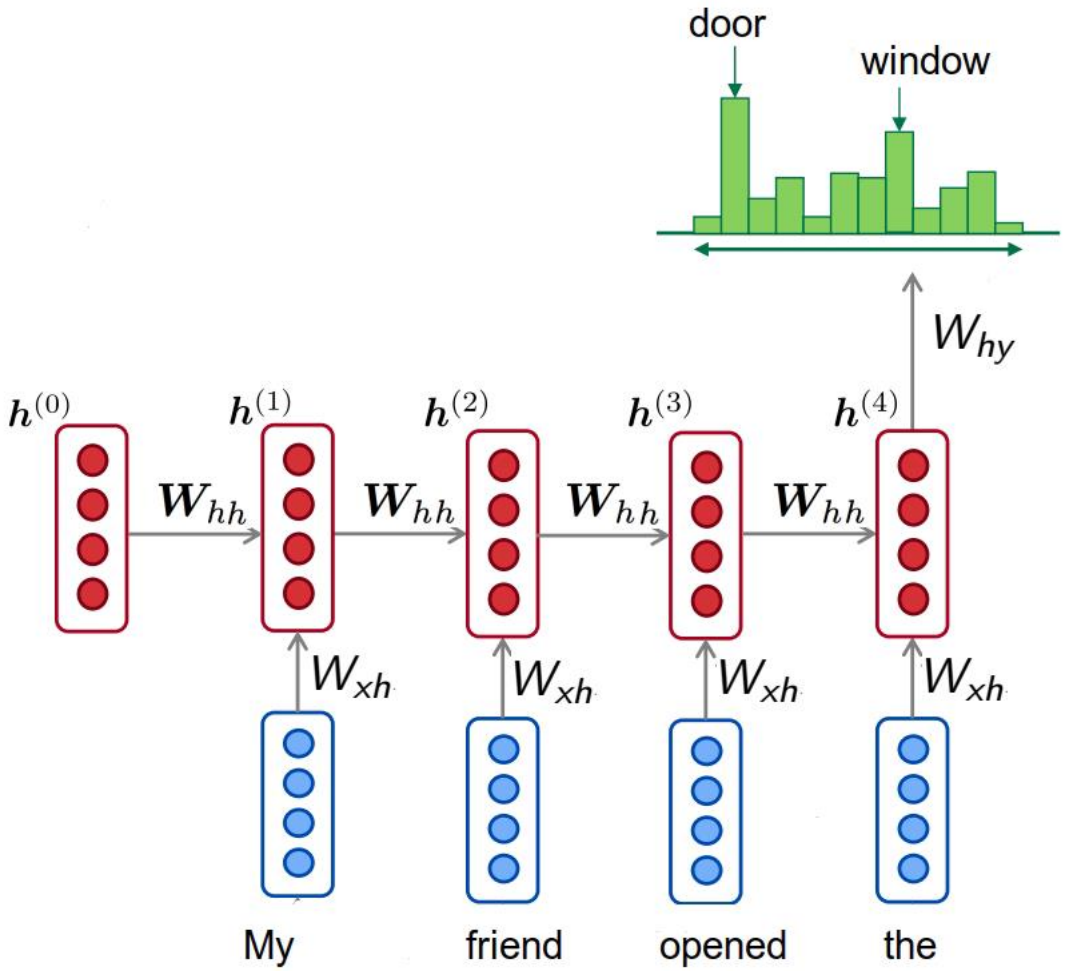
Unfold in time

Go deep

AR Model (RNN)

Network Architectures for Autoregressive Modeling

- **Issues with RNNs**
  - A single hidden vector needs to **summarize all the history**, which can grow extremely large
  - **Sequential** evaluation, can not be parallelized
  - Exploding/vanishing gradients when accessing information from many steps back

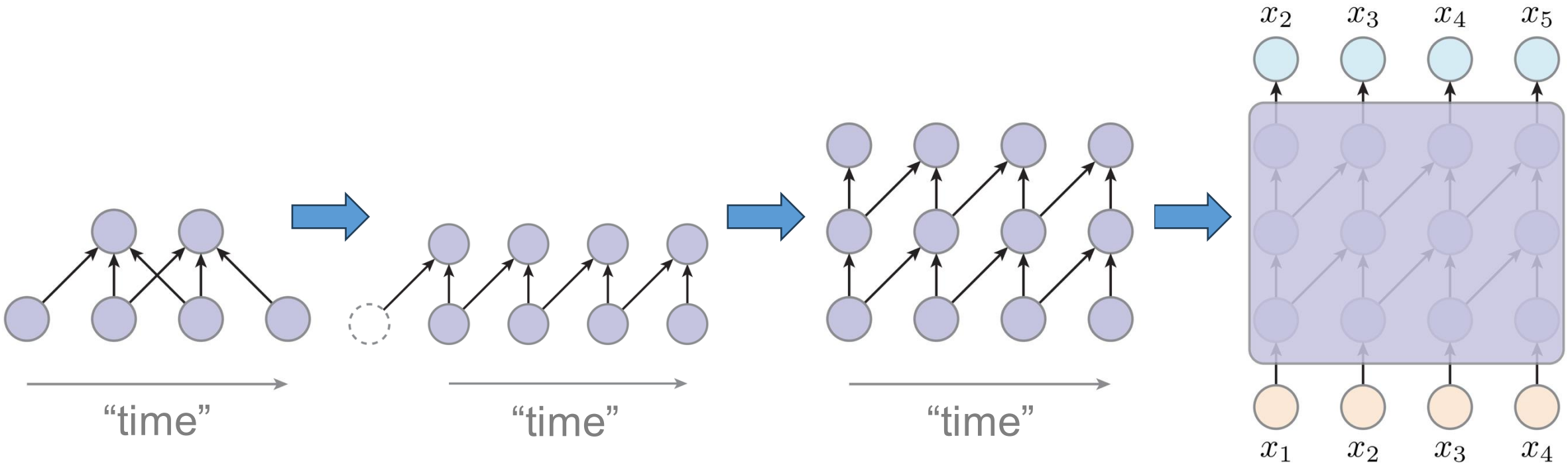


$$h_{t+1} = \tanh(W_{hh}h_t + W_{xh}x_{t+1})$$

Network Architectures for Autoregressive Modeling

- **Common Architectures: CNN**

Shifted by one step



1-D Conv.

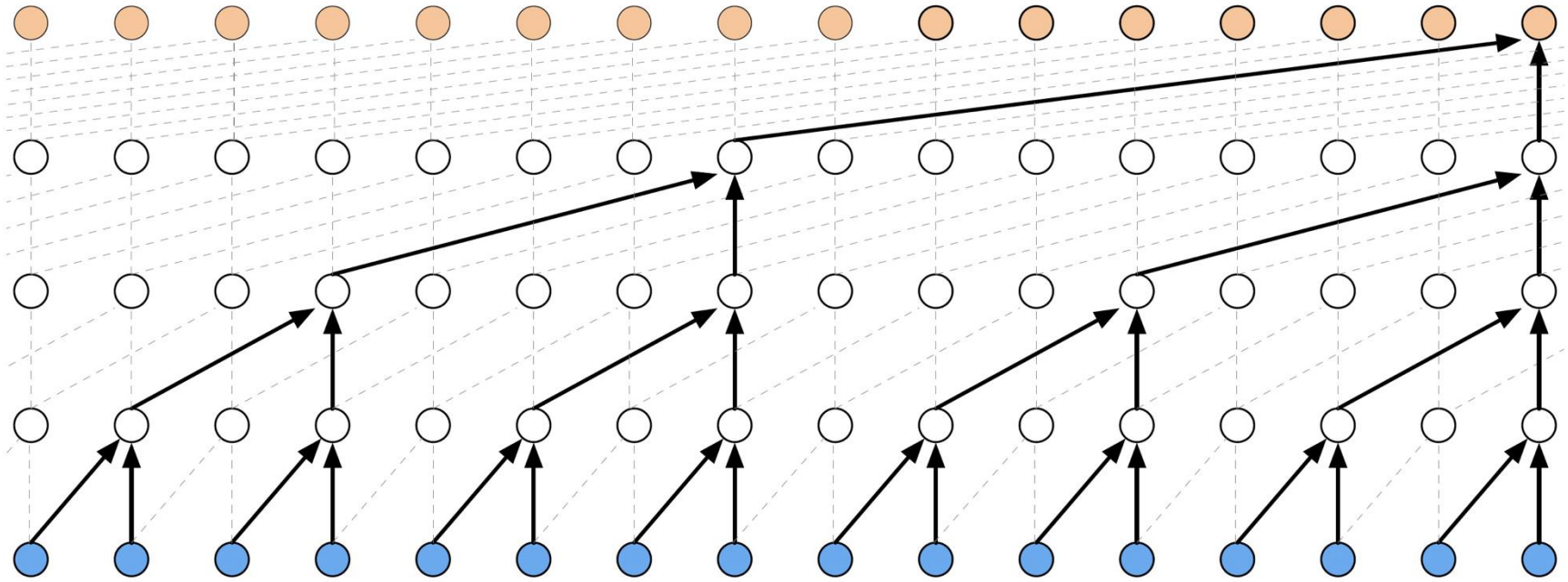
w/ Padding + Casual Conv.

Go deep

AR Model (CNN)

Network Architectures for Autoregressive Modeling

- **CNN-based Autoregressive Model: WaveNet**

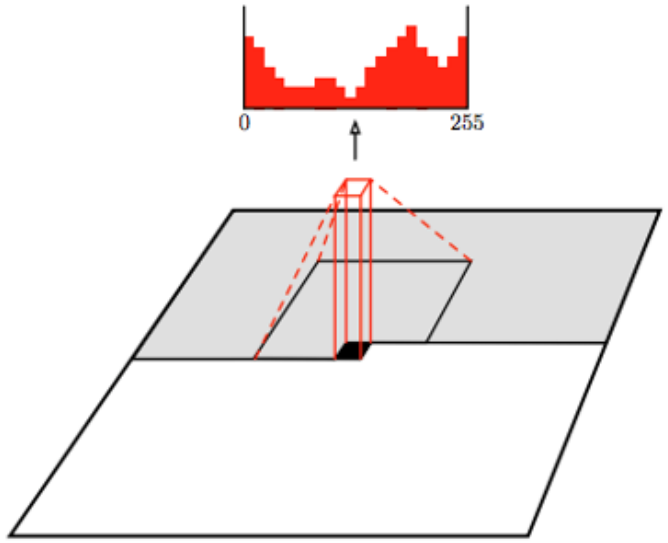


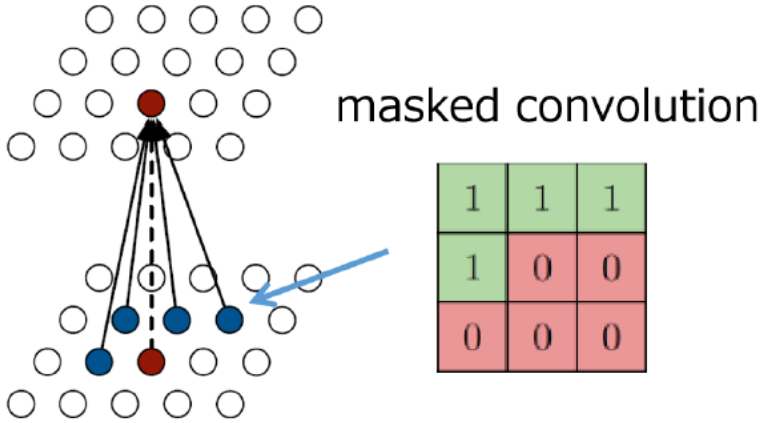
- Audio generation with **causal** and **dilated** 1-D convolutions
- The computational graph looks like a masked autoencoder

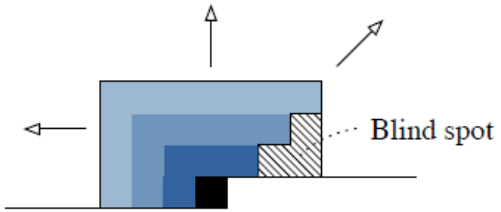
van den Oord, et al. “WaveNet: A Generative Model for Raw Audio”, 2016

Network Architectures for Autoregressive Modeling

- **CNN-based Autoregressive Model: PixelCNN**





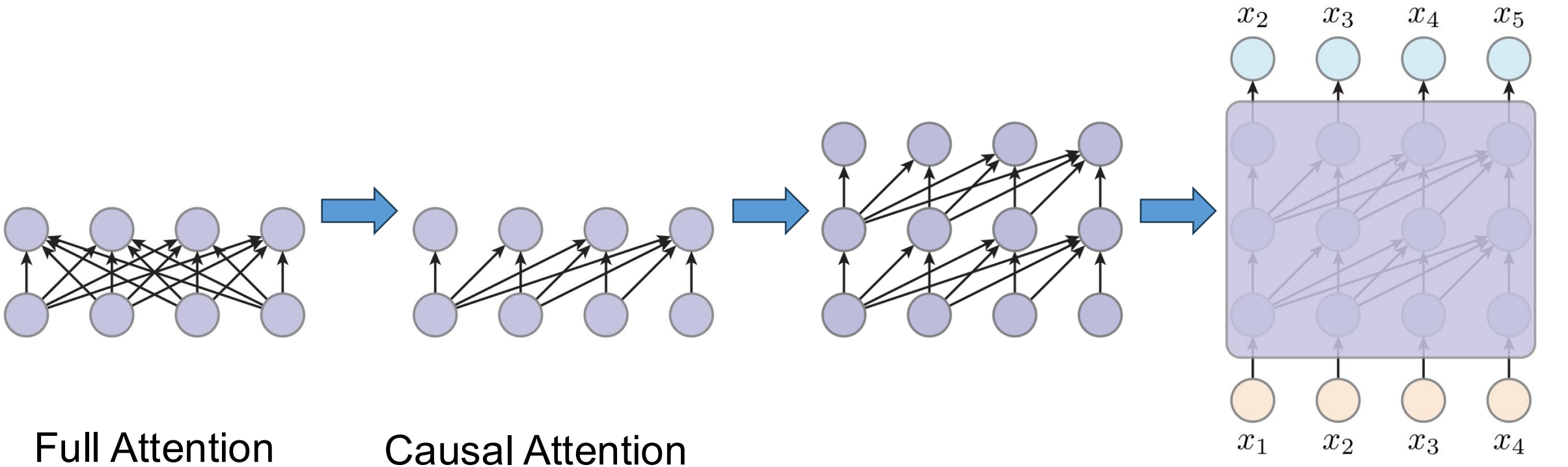


- Use 2D conv. to **predict next pixel given context** (neighborhood of pixels)
- Use **masked convolutions** to ensure autoregressive-ness

van den Oord, et al. “Conditional Image Generation with PixelCNN Decoders”, 2016

Network Architectures for Autoregressive Modeling

- **Common Architectures: Attention**



Full Attention (every step sees all steps)

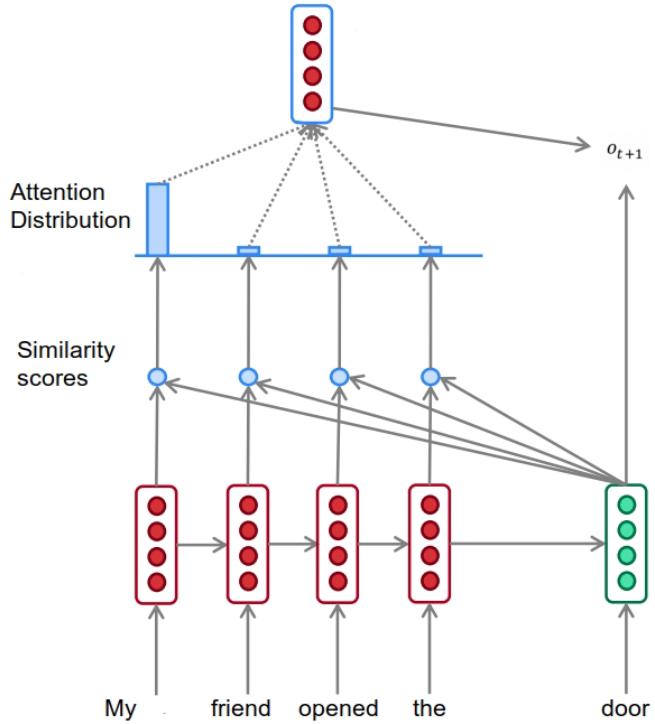
Causal Attention (does not depend on the future)

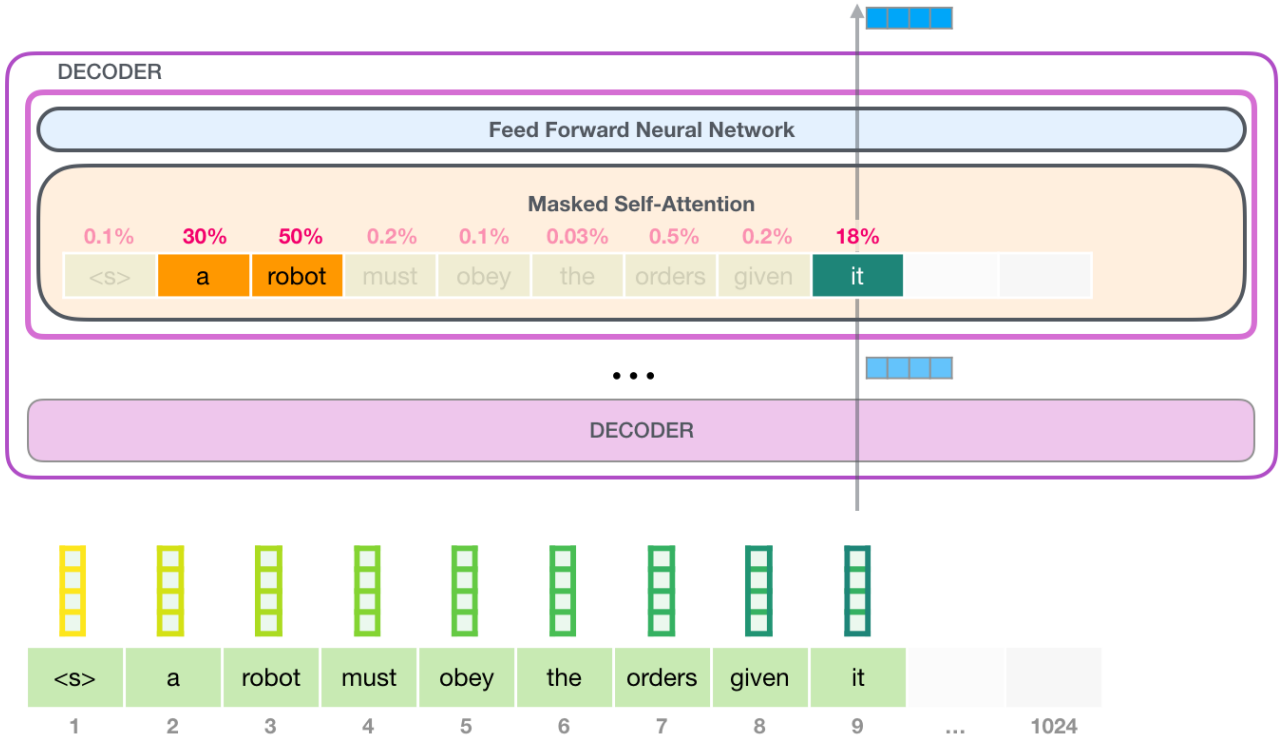
Go deep

AR Model (Attention)

Network Architectures for Autoregressive Modeling

- **Attention-based Autoregressive Model: Transformer & GPT**



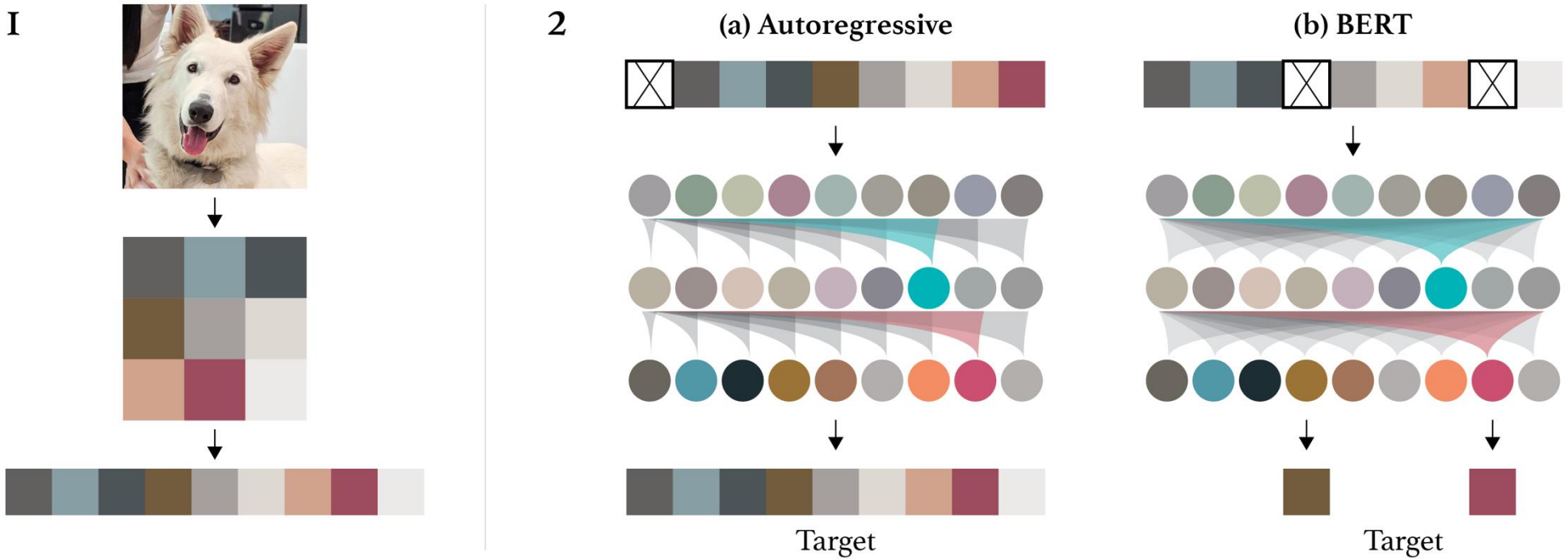


- ➢ Access to all past hidden states, weighted summarize history via attention
- ➢ Use **masked** self-attention to preserve autoregressive structure

Jay Alammar, The Illustrated GPT-2 (Visualizing Transformer Language Models), 2019

Network Architectures for Autoregressive Modeling

- **Attention-based Autoregressive Model: imageGPT**



- ➢ Images are downscaled, flatten, and color-quantized (i.e., token sequence)
- ➢ Use **attention-based network** to model **autoregressive pixel prediction**

OpenAI. "Generative Pretraining from Pixels", 2020



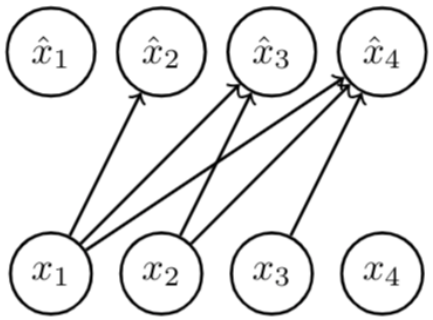
Lecture Outline

- Introduction to Autoregressive Models
- Basic Autoregressive Models
- Network Architectures for Autoregressive Modeling
- Inference and Training Scheme

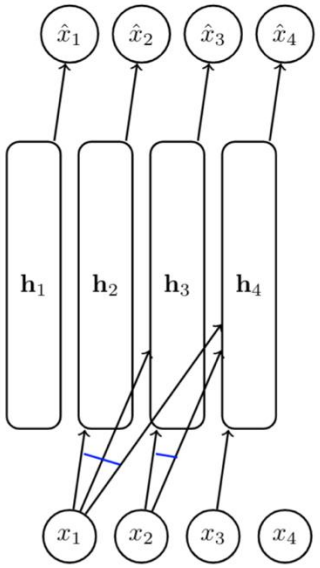
Network Architectures for Autoregressive Modeling

- **Framework Comparison**

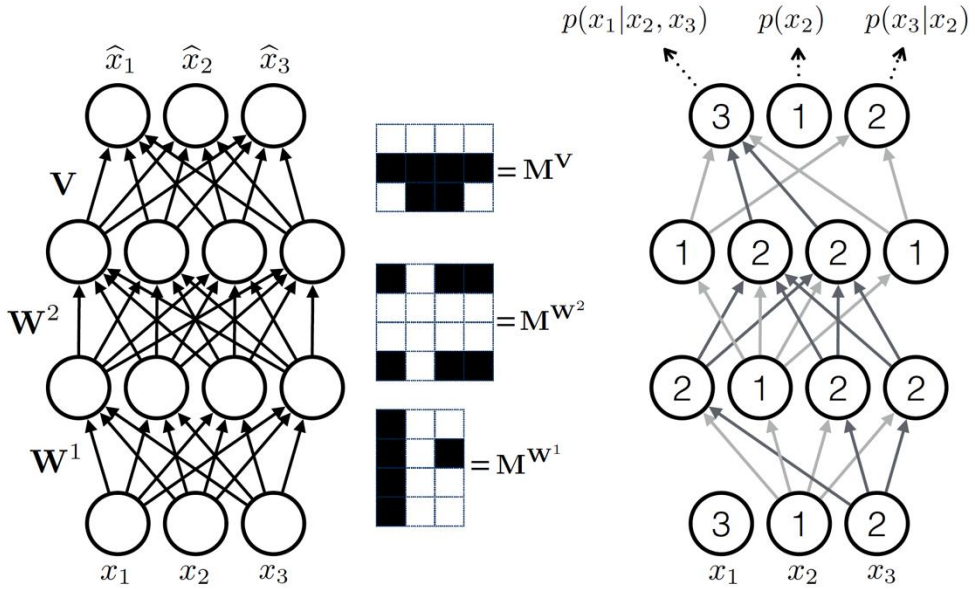
FVSBN



NADE



MADE

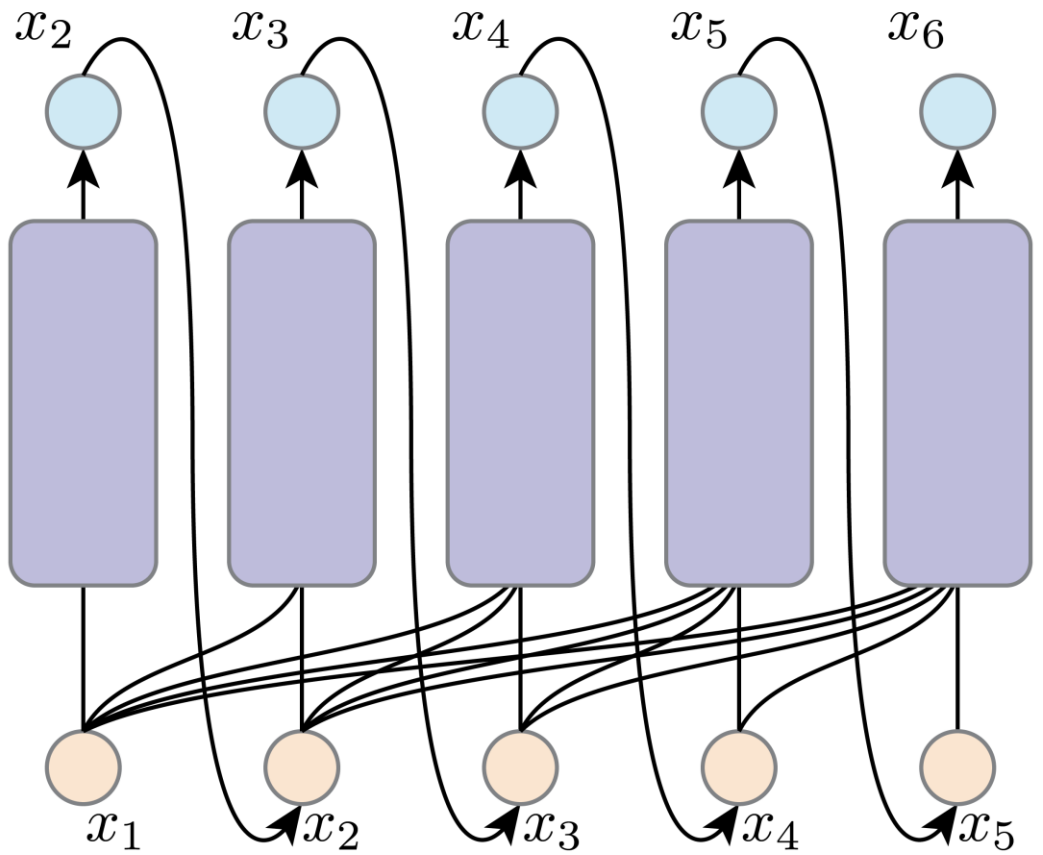


- **Note:** All $\hat{x}_i$ in parallel can be computed in parallel
- **Question:** Are these train-time or inference-time properties?

Inference and Training Scheme

- **Autoregressive Inference**

$$p(x_1, x_2, \dots, x_n) = \prod_{i=1}^{n} p(x_i \mid x_1, x_2, \dots, x_{i-1})$$

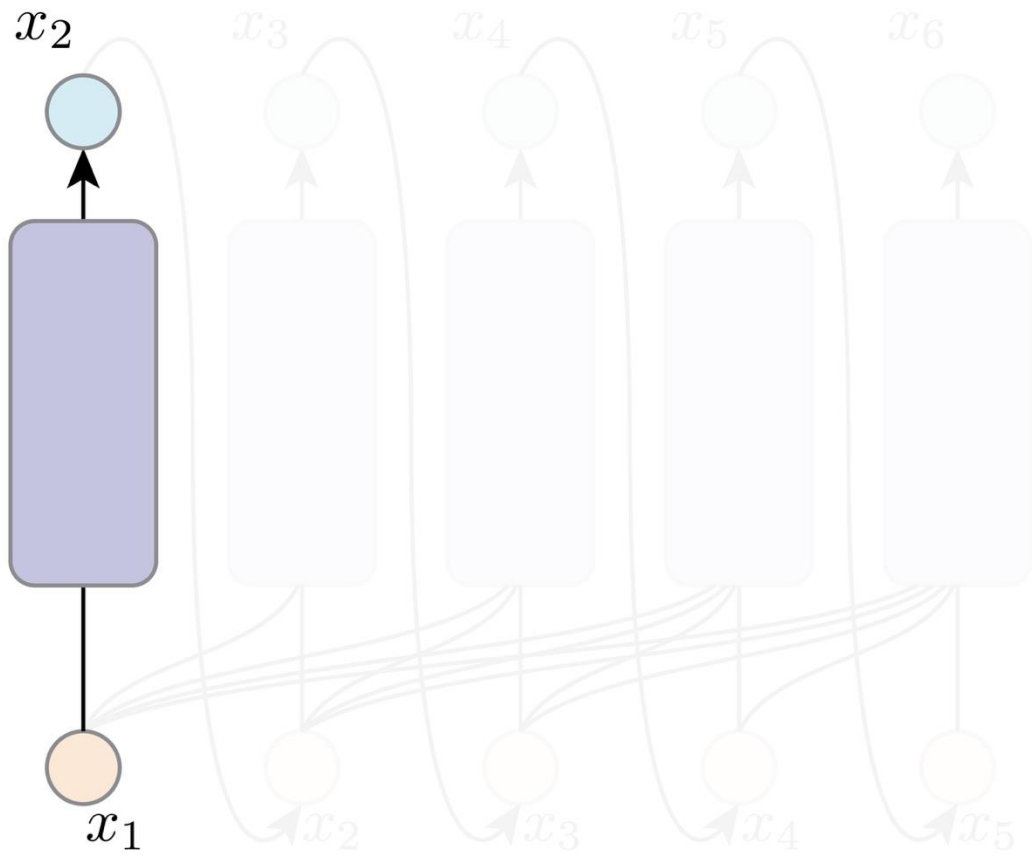


Inference and Training Scheme

- **Autoregressive Inference**
  - ➢ This **network** models

$$p(x_2 \mid x_1)$$

  - ✓ 1 input
  - ✓ 1 output

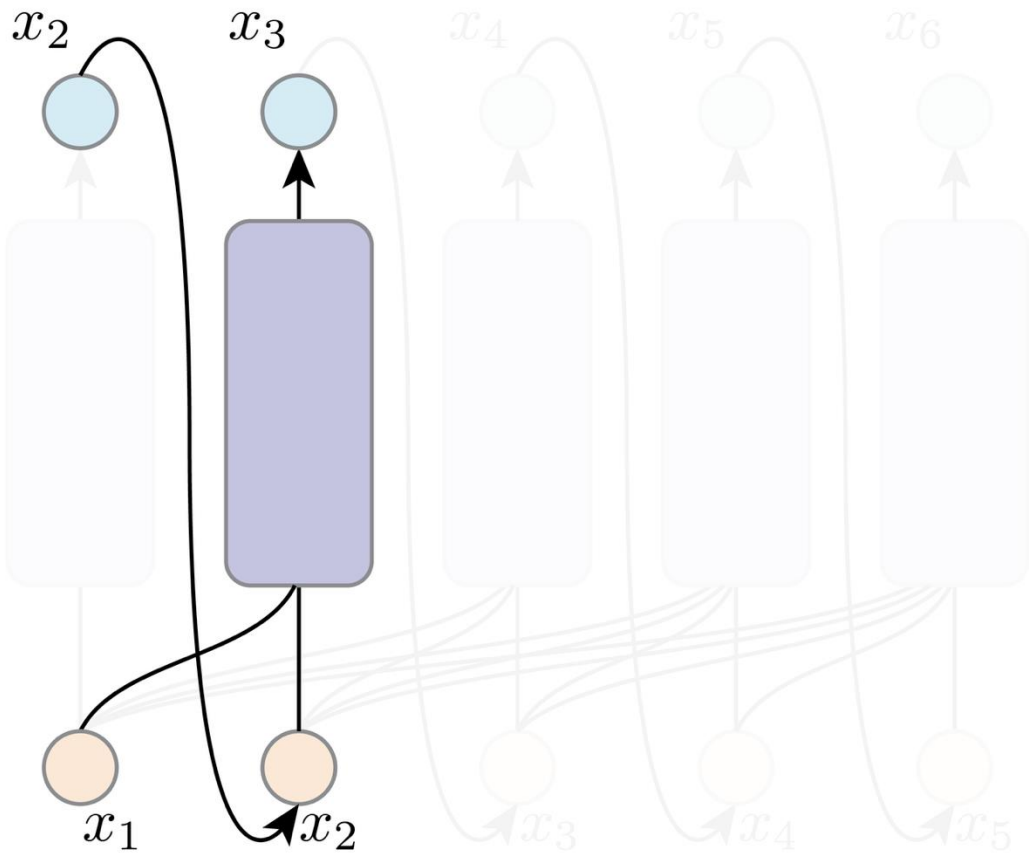


Inference and Training Scheme

- **Autoregressive Inference**
  - This **network** models

$$p(x_3 \mid x_1, x_2)$$

    - ✓ **2 inputs**
    - ✓ 1 output
  - **Note:** inputs are outputs from previous steps

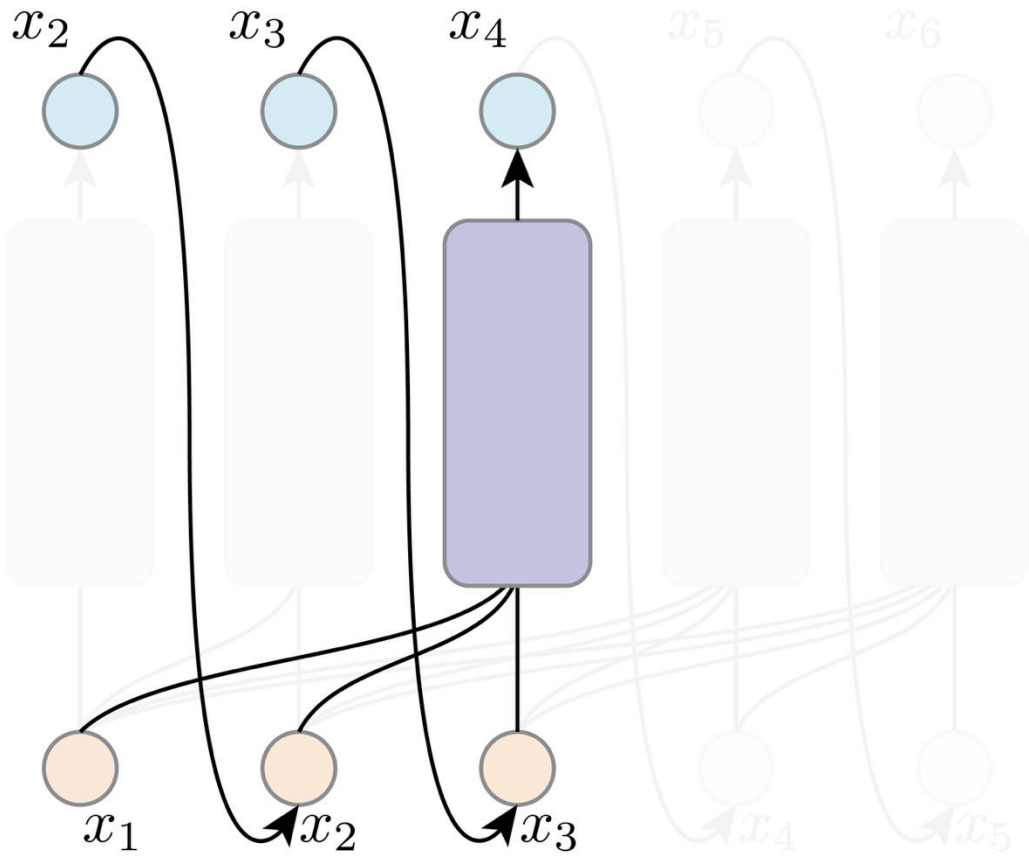


Inference and Training Scheme

- **Autoregressive Inference**
  - ➢ This **network** models

    $$p(x_4 \mid x_1, x_2, x_3)$$

    - ✓ **3 inputs**
    - ✓ 1 output
  - ➢ **Note:** inputs are outputs from previous steps



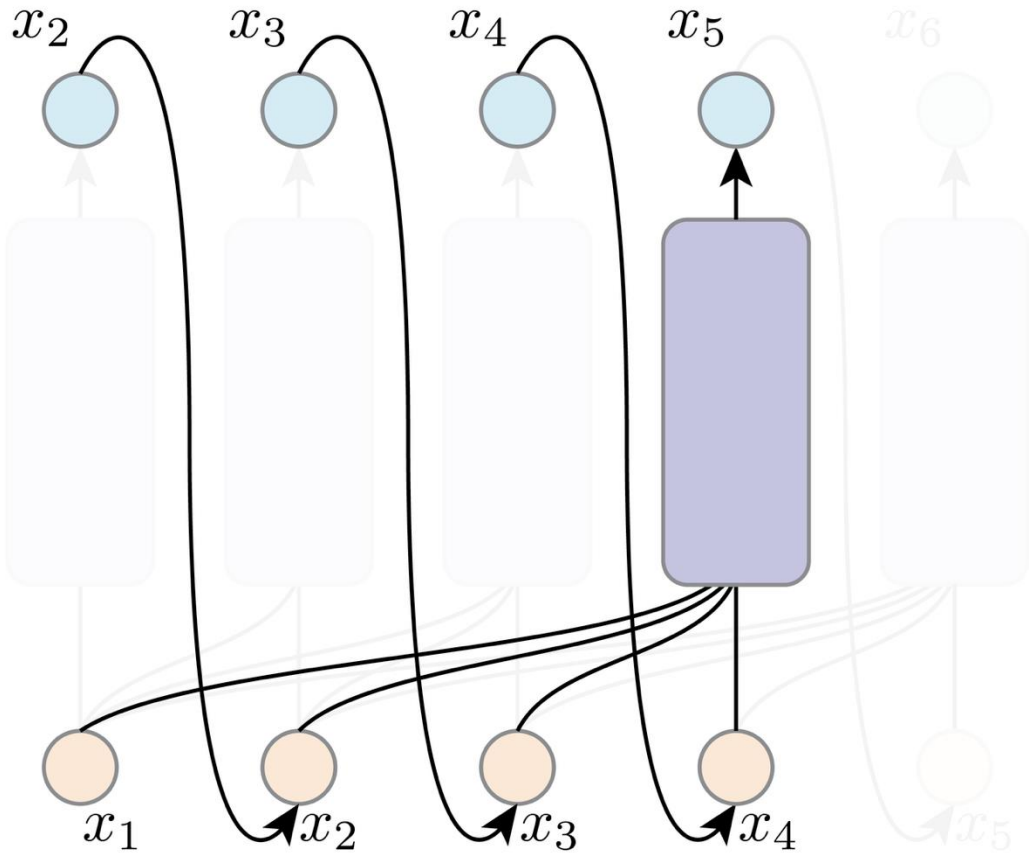
Inference and Training Scheme

- **Autoregressive Inference**
  - This **network** models

    $p(x_5 \mid x_1, x_2, x_3, x_4)$

    ✓ **4 inputs**

    ✓ 1 output
  - **Note:** inputs are outputs from previous steps



Inference and Training Scheme

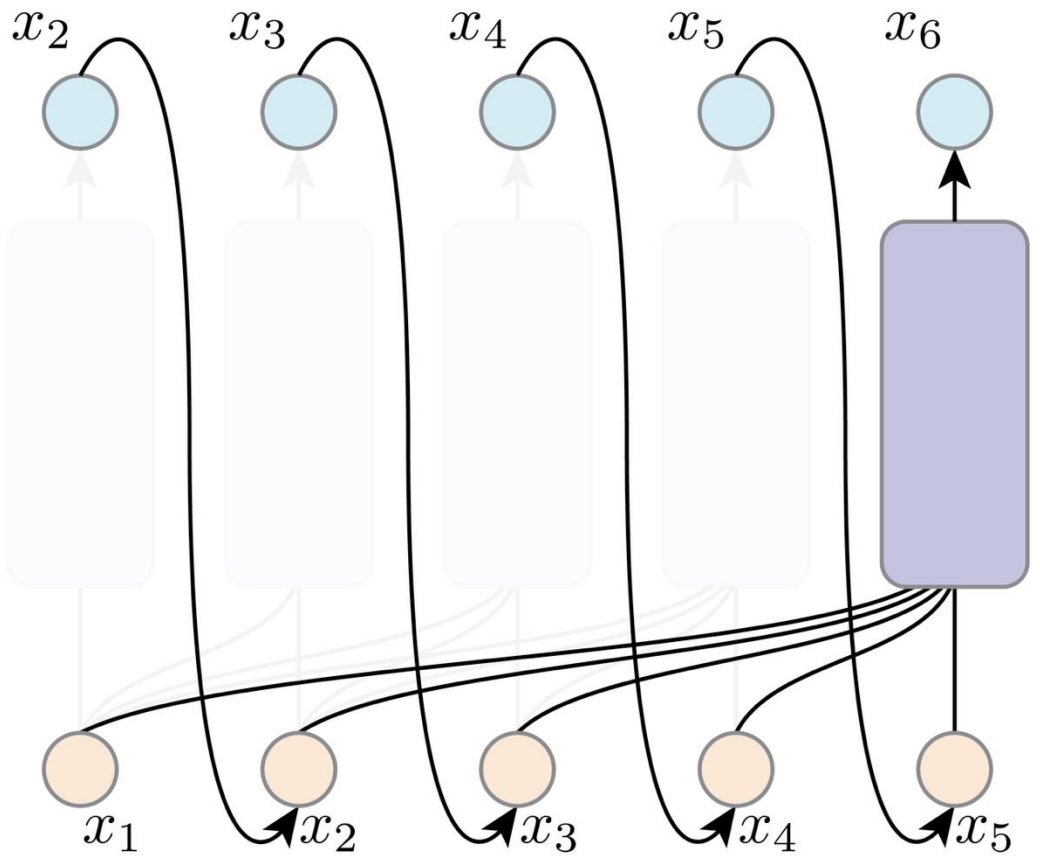
- **Autoregressive Inference**
  - ➢ This **network** models

    $p(x_6 \mid x_1, x_2, x_3, x_4, x_5)$

    ✓ **5 inputs**

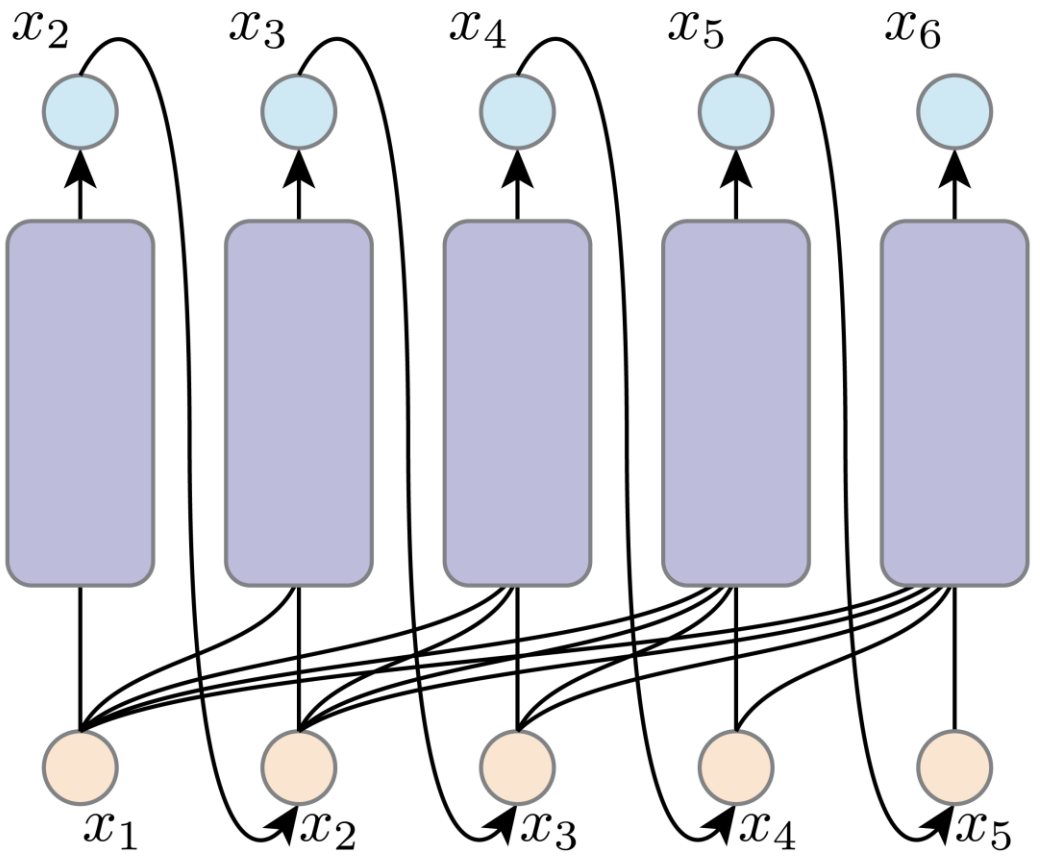
    ✓ 1 output

  - ➢ **Note:** inputs are outputs from previous steps



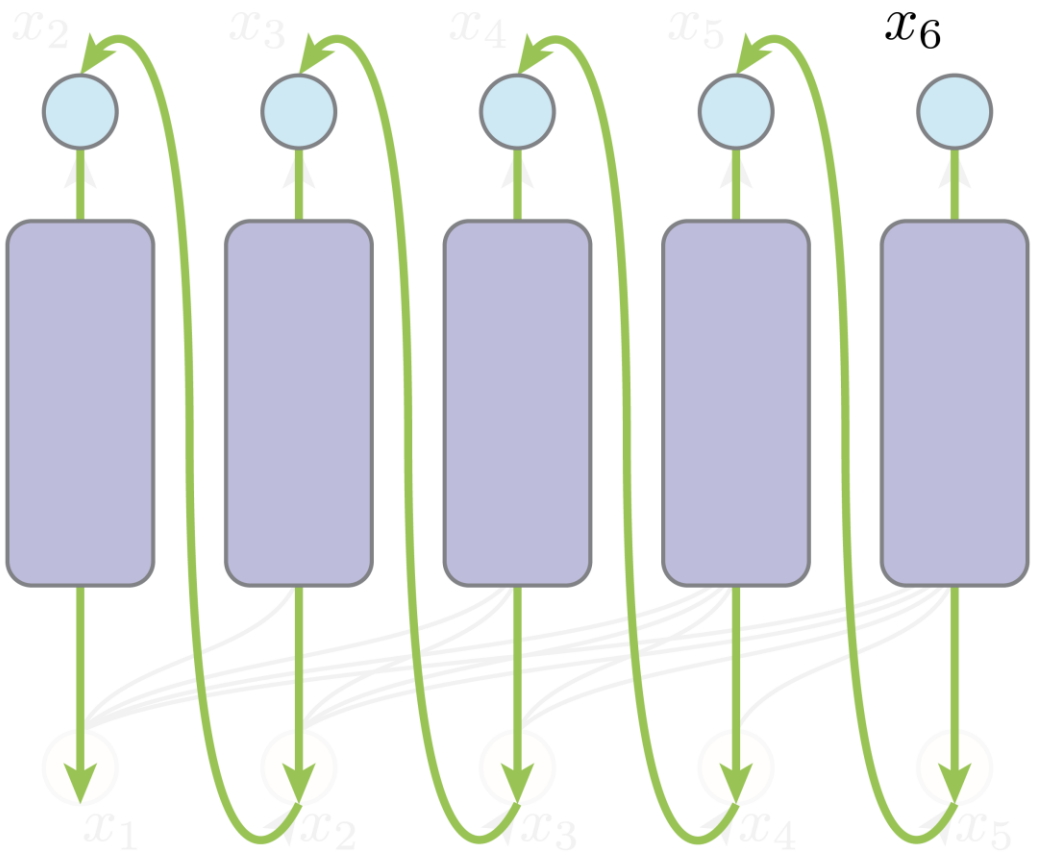
Inference and Training Scheme

- **Autoregressive Inference**
  - ➢ This is a **recursive** process
  - ➢ **Note:**
    - ✓ **Not** necessarily RNN
    - ✓ Can be done by **any** architecture (CNN or Transformer)



Inference and Training Scheme

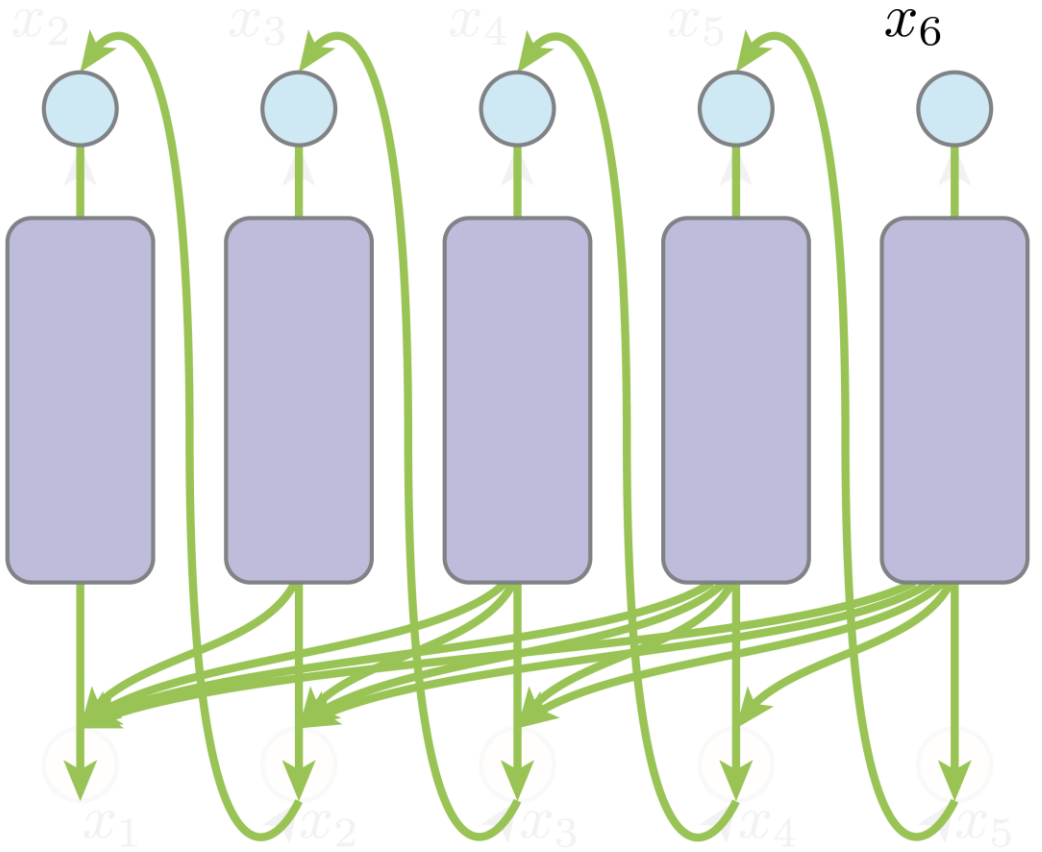
- **Backpropagation?**
  - ➢ Consider the **gradient path** of x_6
  - ➢ Go through …
    - ✓ all previous outputs
    - ✓ all previous operators
    - ✓ all previous networks (each is a deep model)
- **Infeasible to train the AR model following its inference graph**



Single inference path

Inference and Training Scheme

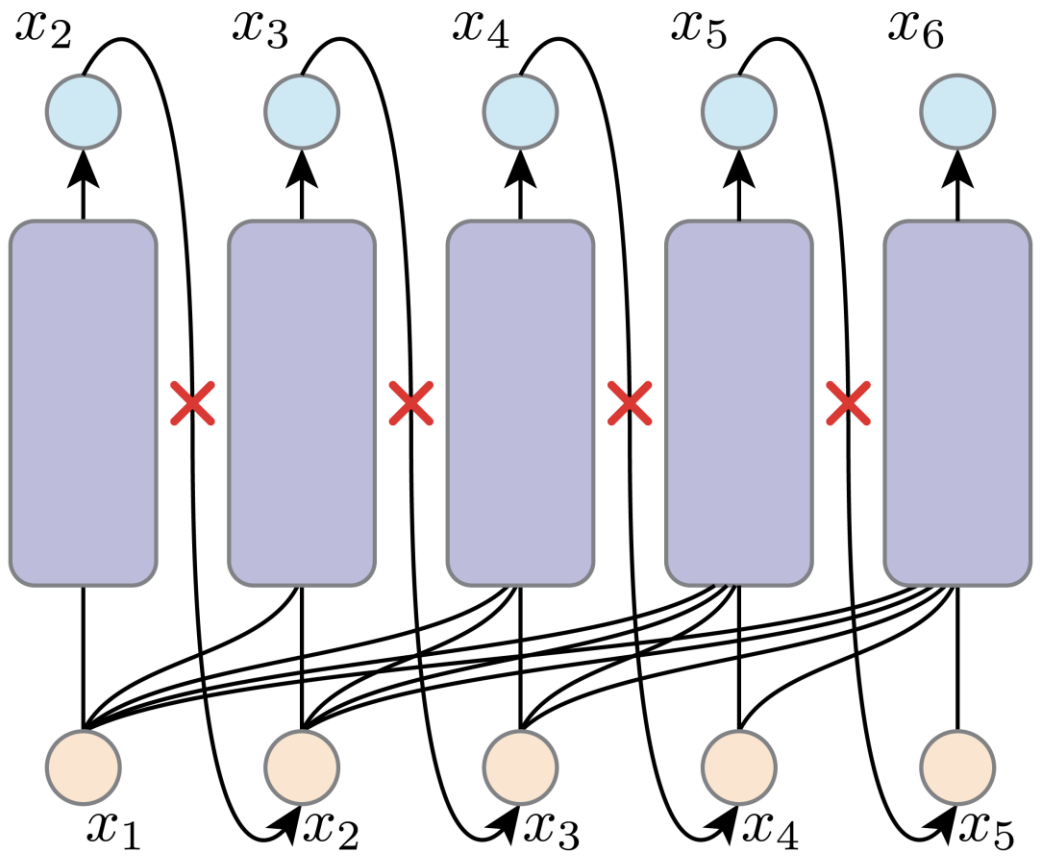
- **Backpropagation?**
  - ➢ Consider the **gradient path** of x_6
  - ➢ Go through …
    - ✓ all previous outputs
    - ✓ all previous operators
    - ✓ all previous networks (each is a deep model)
- **Infeasible to train the AR model following its inference graph**



Full gradient paths

Inference and Training Scheme

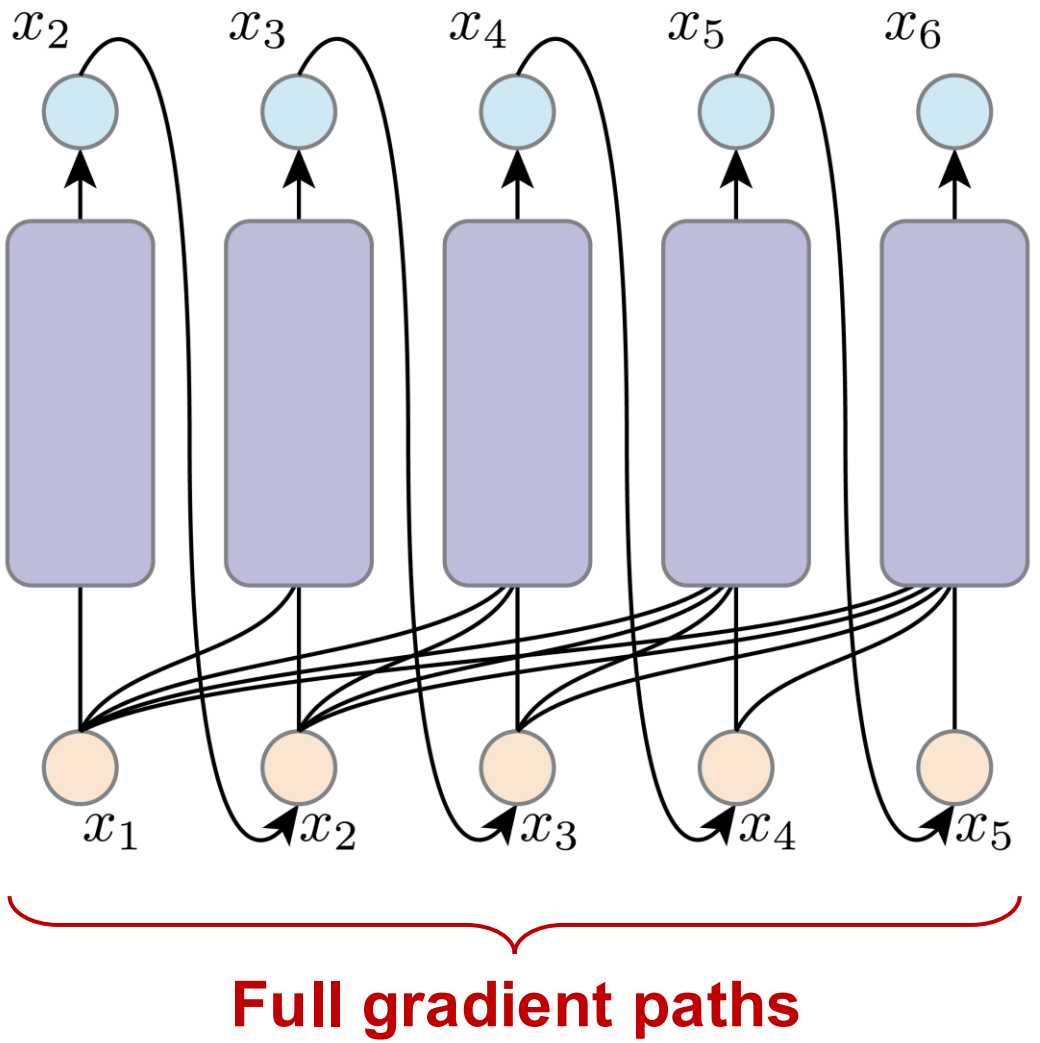
- **Teacher Forcing**
  - Inputs are not previous outputs
  - Inputs are from ground-truth data



Full gradient paths

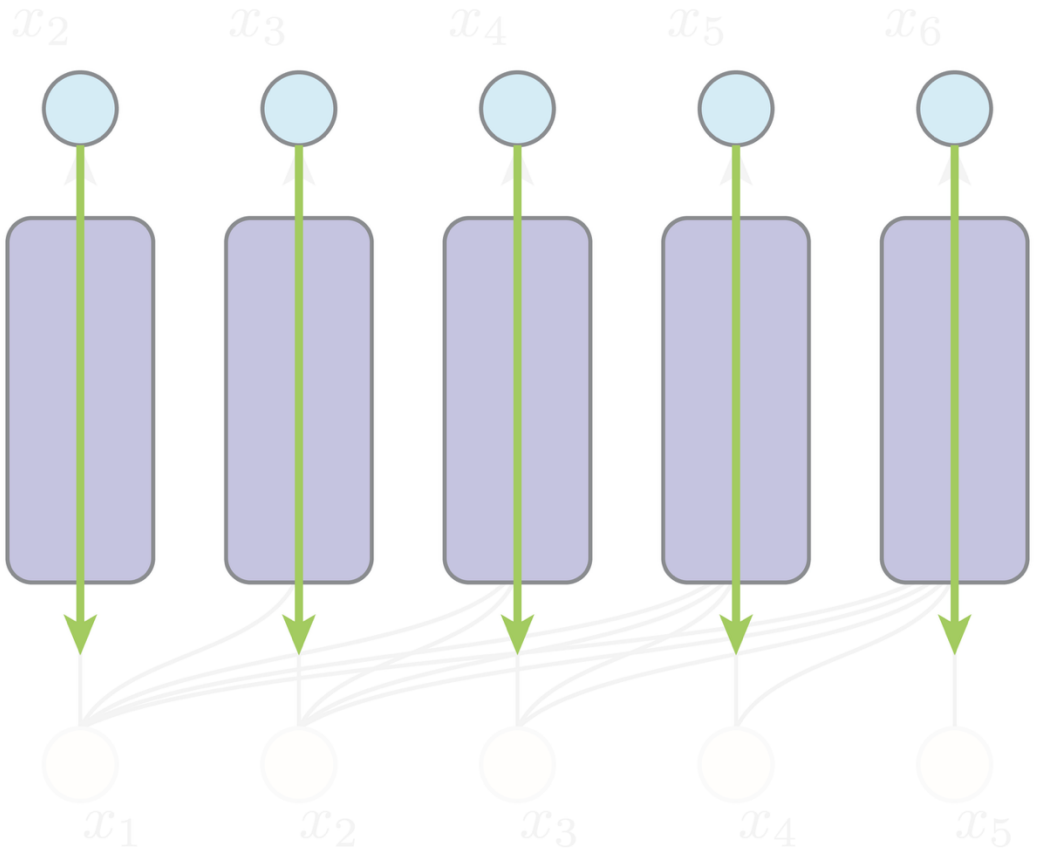
Inference and Training Scheme

- **Teacher Forcing**
  - Inputs are not previous outputs
  - Inputs are from ground-truth data



Inference and Training Scheme

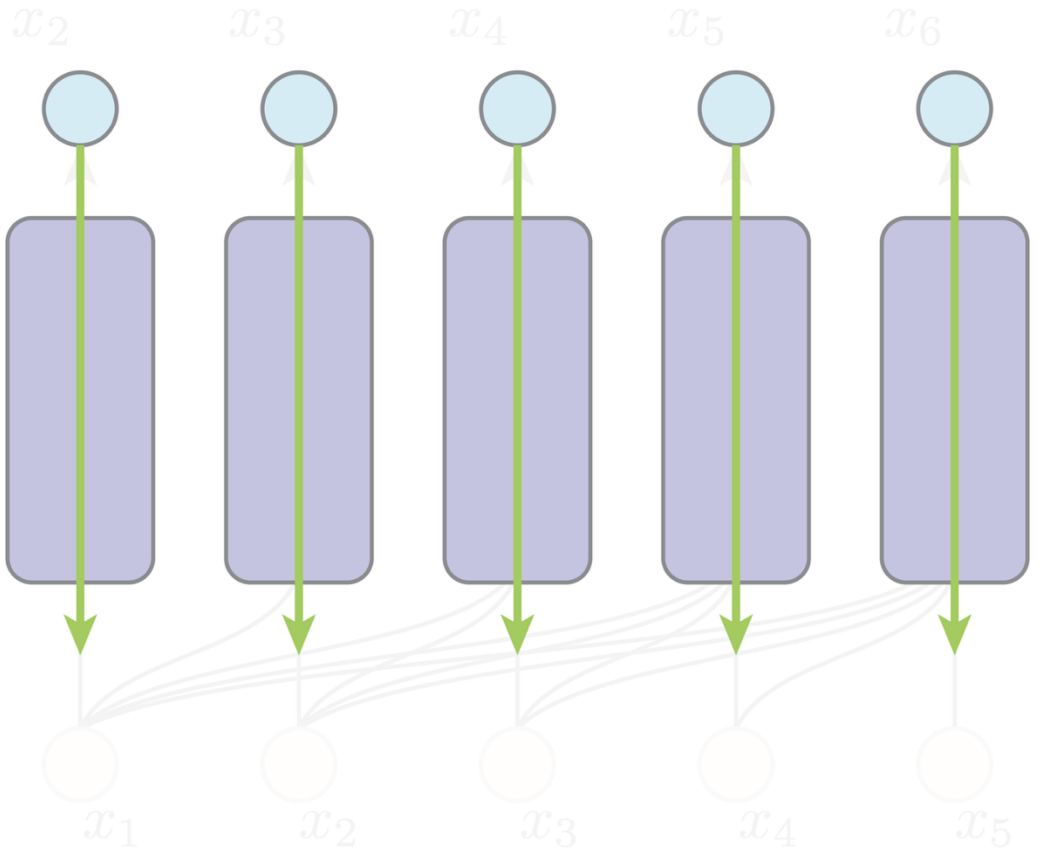
- **Teacher Forcing**
  - Inputs are not previous outputs
  - Inputs are from ground-truth data
- **Pros**
  - BP path is much shorter
  - Training is very much eased



Each path is a single deep model

Inference and Training Scheme

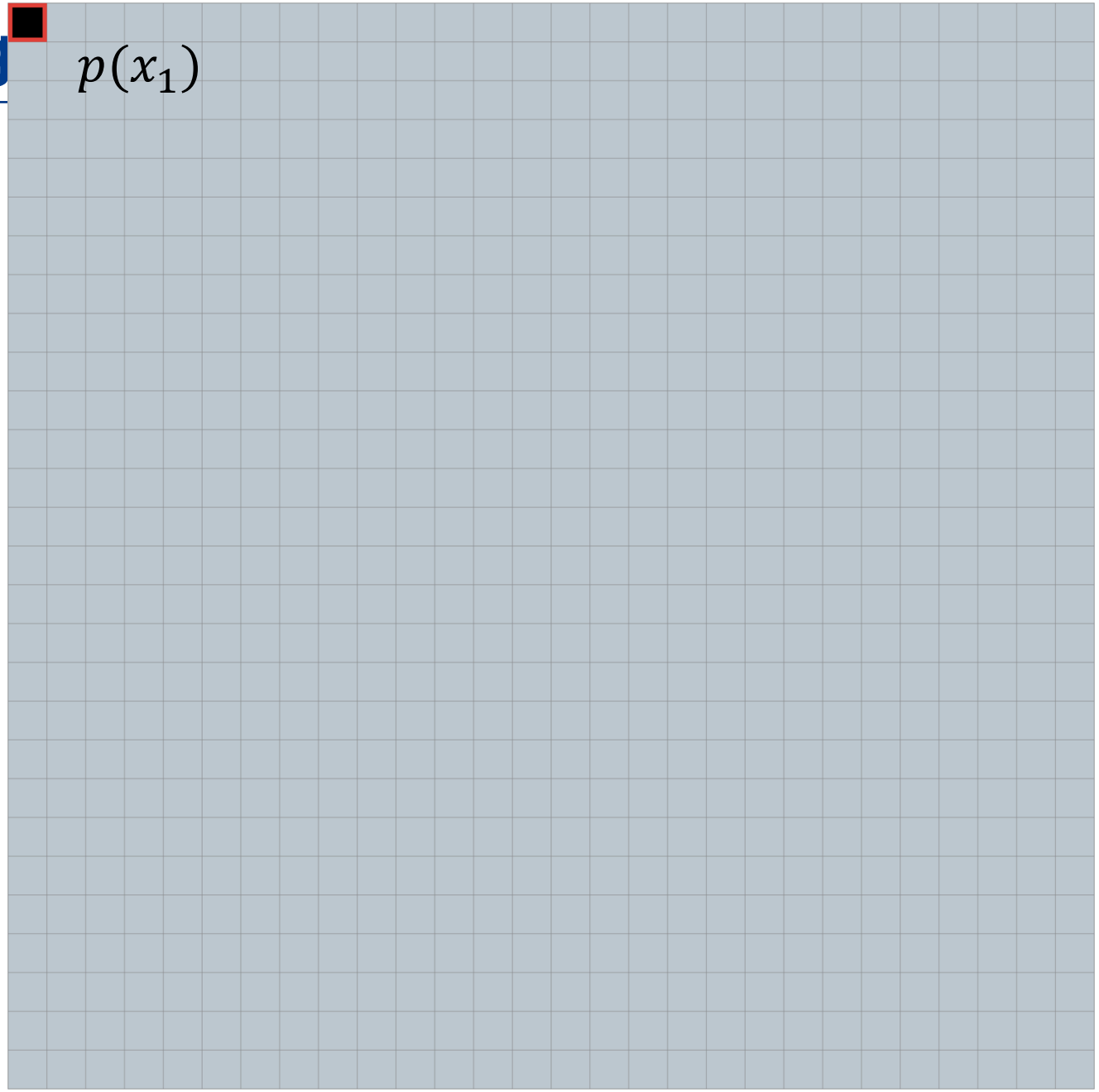
- **Teacher Forcing**
  - Inputs are not previous outputs
  - Inputs are from ground-truth data
- **Pros**
  - BP path is much shorter
  - Training is very much eased
- **Cons**
  - **Train – Inference mis-match**
  - **Distribution shift (exposure bias)**



Each path is a single deep model

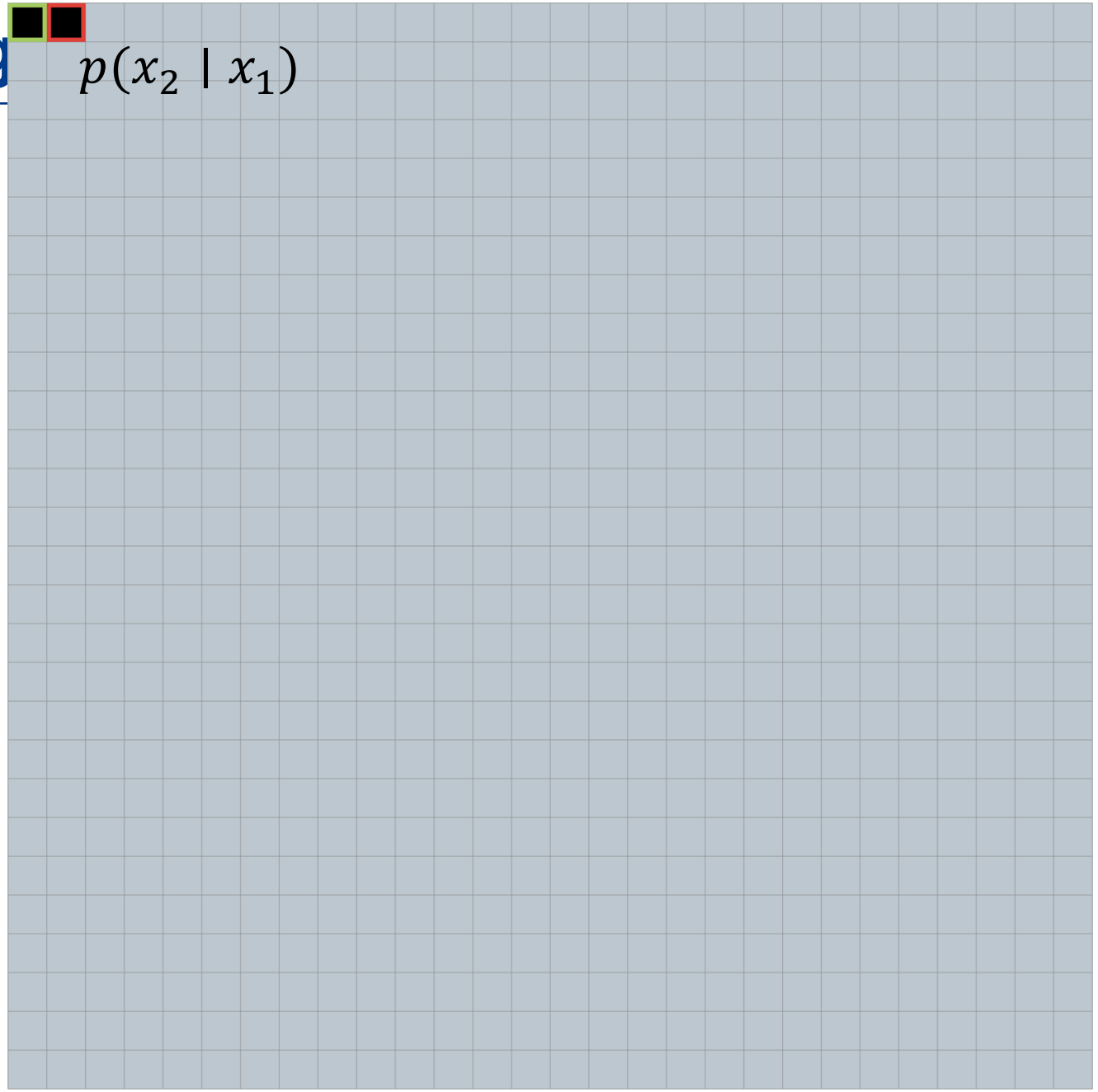
Inference and Training

- **Inference: Autoregressive**
  - Sample this from $p(x_1)$



Inference and Training

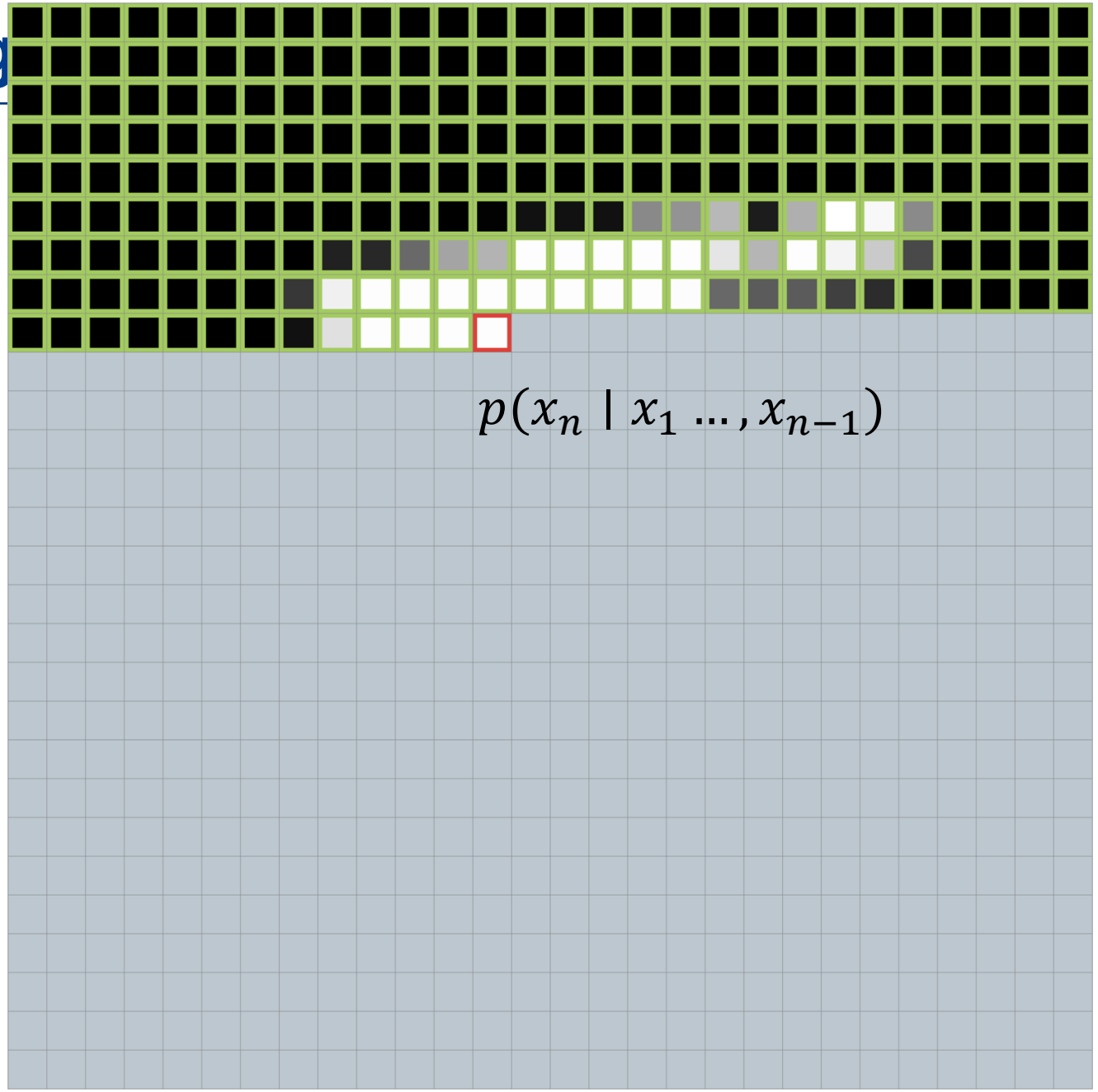
- **Inference: Autoregressive**
  - Sample this from $p(x_2 \mid x_1)$
  - This is output from previous step, input for current step
  - Network for current step
    - ✓ 1 input
    - ✓ 1 predict



Inference and Training

- **Inference: Autoregressive**
  - Sample this from

    $p(x_n \mid x_1 \ldots, x_{n-1})$
  - These are outputs from previous step, input for current step
  - Network for current step
    - ✓ n-1 inputs
    - ✓ 1 predict

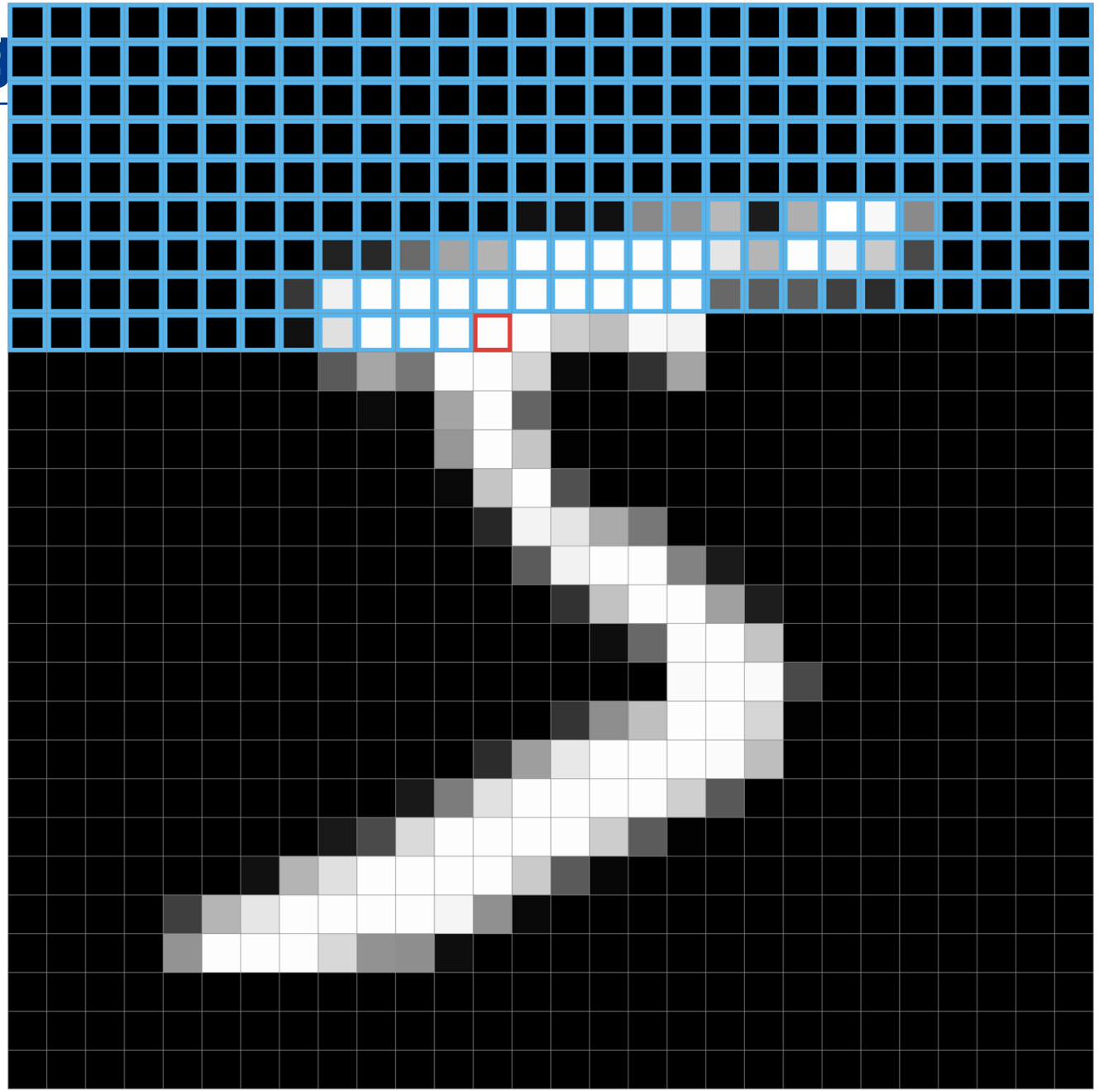


Inference and Training

- **Training: Teacher-Forcing**
  - Model this from

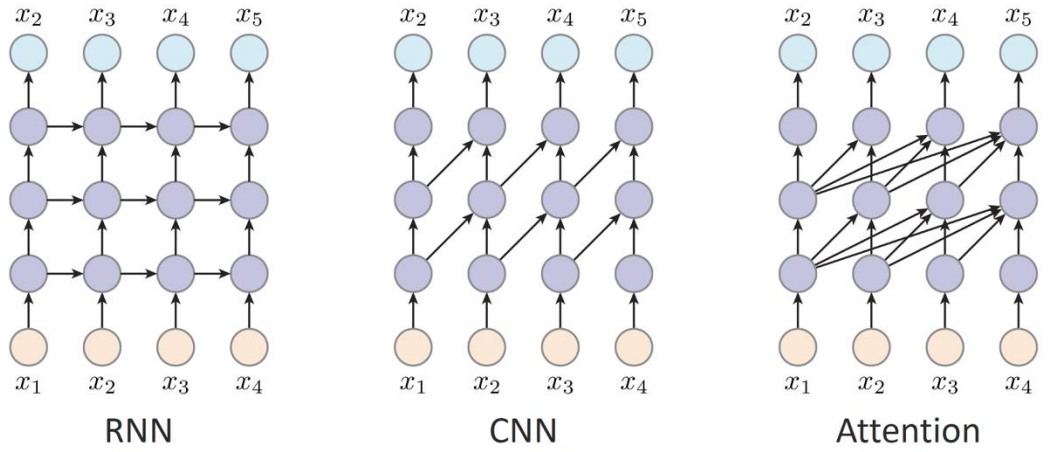
    $$p(x_n \mid x_1 \ldots, x_{n-1})$$

  - These are outputs from **ground-truth**, input for current step
  - Network for current step
    - ✓ n-1 inputs
    - ✓ 1 predict



Summary

- **Introduction to Autoregressive Models**
  - AR-ness + Inductive Bias $p(x_1, x_2, \ldots, x_n) = \prod_{i=1}^{n} p(x_i \mid x_1, x_2, \ldots, \boldsymbol{x_{<i}})$
- **Basic Autoregressive Models**
  - AR models: Autoencoder + Masked weights
- **Network Architectures for Autoregressive Modeling**
  - RNN, CNN, Attention
- **Inference and Training Scheme**
  - Teacher Forcing



Supplementary Material

- **Stanford CS 236 Lecture 3**
  - https://www.youtube.com/watch?v=rNEujZmD2Tg&list=PLoROMvodv4rPOWA-omMM6STXaWW4FvJT8&index=3
- **Cornell CS 6785 Lecture 3**
  - https://www.youtube.com/watch?v=FBGj_B6hH9Y&list=PL2UML_KCiC0UPzjW9BjO-lW6dqliu9O4B&index=3
- **Deep Generative Models**
  - **Section 3**

Reading List

- **AR Models:**
  - Y. Bengio and S. Bengio, Modeling High-Dimensional Discrete Data with Multi-Layer Neural Networks, NIPS 1999
  - A. Van Den Oord et al., Pixel Recurrent Neural Networks, ICML 2016
  - D. P. Kingma et al., Improved Variational Inference with Inverse Autoregressive Flow, NIPS 2016
- **AR Models & Tokenizers:**
  - L. Yu et al., Language Model Beats Diffusion -- Tokenizer is Key to Visual Generation, ICLR 2024
  - K. Tian et al., Visual Autoregressive Modeling: Scalable Image Generation via Next-Scale Prediction, arXiv
  - Q. Yu et al., An Image is Worth 32 Tokens for Reconstruction and Generation, arXiv

Reading List

- **AR Models & Tokenizers (Cont.):**
  - M. Chen et al., Generative Pretraining from Pixels, ICML 2020
  - A. Ramesh et al., Zero-Shot Text-to-Image Generation, ICML 2021
  - F. Mentzer et al., Finite Scalar Quantization: VQ-VAE Made Simple, arXiv
- **AR Models & Diffusion:**
  - T. Li et al., Autoregressive Image Generation without Vector Quantization,
  - C. Zhou et al., Transfusion: Predict the Next Token and Diffuse Images with One Multi-Modal Model, arXiv
  - J. Xie et al., Show-o: One Single Transformer to Unify Multimodal Understanding and Generation, arXiv
  - E. Hoogeboom et al., Autoregressive Diffusion Models, ICLR 2022
  - B. Chen et al., Diffusion Forcing: Next-token Prediction Meets Full-Sequence Diffusion, arXiv